——— backend/models.py ———

```
# -*- coding: utf-8 -*-
"""
求职备考管理软件 V1.0
数据访问层：基于 SQLite 的全部数据表定义与 CRUD 操作。

本模块负责：
  1. 数据库文件初始化与表结构创建（含版本升级兼容）
  2. 用户账号（单用户本地登录）的密码哈希与校验
  3. 六大业务实体（求职投递 / 笔试练习 / 备考资料 / 待办事项 / 考试日程 / 操作日志）
     的增删改查与条件查询
  4. 为统计模块提供聚合查询原语

设计原则：
  - 零外部依赖，仅使用 Python 标准库 sqlite3 / hashlib / json / datetime
  - 所有写操作均在事务中完成，异常时回滚
  - 时间统一使用 ISO8601 字符串（YYYY-MM-DD 或 YYYY-MM-DD HH:MM:SS），便于排序与展示
"""

import hashlib
import json
import os
import sqlite3
import time
from datetime import datetime


# ---------------------------------------------------------------------------
# 常量定义：业务字典（与前端下拉选项保持一致）
# ---------------------------------------------------------------------------

# 求职投递所处阶段（按求职流程先后排序）
JOB_STAGES = ["待投递", "已投递", "笔试", "面试", "Offer", "已拒绝"]

# 笔试练习模块（覆盖行测主要题型）
PRACTICE_MODULES = ["图形推理", "逻辑判断", "数量分析", "资料分析", "言语理解", "常识判断"]

# 备考资料分类
MATERIAL_CATEGORIES = ["行测", "申论", "专业课", "面试", "简历", "其他"]

# 备考资料学习状态
MATERIAL_STATUS = ["未开始", "学习中", "已完成"]

# 待办事项分类
TODO_CATEGORIES = ["求职", "备考", "生活", "其他"]

# 待办优先级
TODO_PRIORITIES = ["高", "中", "低"]

# 数据库结构版本：用于未来升级时做迁移判断
SCHEMA_VERSION = 1


# ---------------------------------------------------------------------------
# 底层连接管理
# ---------------------------------------------------------------------------

class Database:
    """SQLite 连接封装：负责建表、升级与连接复用。"""

    def __init__(self, db_path):
        self.db_path = db_path
```

```
    # 目录不存在时自动创建，允许把数据库放在任意工作目录下
    parent = os.path.dirname(os.path.abspath(db_path))
    if parent and not os.path.exists(parent):
        os.makedirs(parent, exist_ok=True)
    self._conn = None
    self._init_schema()

def connect(self):
    """获取数据库连接。

    Flask 开发服务器会为每个请求起新线程，而 sqlite3 连接默认绑定
    创建线程；此处关闭该检查（本软件为单用户本地应用，无并发写
    竞争），连接在进程内复用。
    """
    if self._conn is None:
        self._conn = sqlite3.connect(self.db_path,
                                     check_same_thread=False)
        # 返回字典型行，便于直接序列化为 JSON
        self._conn.row_factory = sqlite3.Row
    return self._conn

def close(self):
    if self._conn is not None:
        self._conn.close()
        self._conn = None

# -- 表结构 ------------------------------------------------------------
def _init_schema(self):
    conn = self.connect()
    cur = conn.cursor()

    # 用户表：单用户本地账号，密码加盐哈希存储
    cur.execute("""
        CREATE TABLE IF NOT EXISTS users (
            id            INTEGER PRIMARY KEY AUTOINCREMENT,
            username      TEXT NOT NULL UNIQUE,
            password_hash TEXT NOT NULL,
            salt          TEXT NOT NULL,
            created_at    TEXT NOT NULL
        )
    """)

    # 求职投递表：记录每一次网申的全生命周期
    cur.execute("""
        CREATE TABLE IF NOT EXISTS job_applications (
            id           INTEGER PRIMARY KEY AUTOINCREMENT,
            company      TEXT NOT NULL,
            position     TEXT NOT NULL,
            city         TEXT DEFAULT '',
            stage        TEXT NOT NULL DEFAULT '待投递',
            source       TEXT DEFAULT '',
            applied_date TEXT DEFAULT '',
            deadline     TEXT DEFAULT '',
            salary_range TEXT DEFAULT '',
            priority     TEXT DEFAULT '中',
            notes        TEXT DEFAULT '',
            created_at   TEXT NOT NULL,
            updated_at   TEXT NOT NULL
        )
    """)

    # 笔试练习记录表：每一次刷题练习的量化记录
```

```
cur.execute("""
    CREATE TABLE IF NOT EXISTS practice_sessions (
        id            INTEGER PRIMARY KEY AUTOINCREMENT,
        module        TEXT NOT NULL,
        question_count INTEGER NOT NULL,
        correct_count INTEGER NOT NULL,
        duration_min  REAL DEFAULT 0,
        practiced_at  TEXT NOT NULL,
        notes         TEXT DEFAULT '',
        created_at    TEXT NOT NULL
    )
""")

# 备考资料表：书籍 / 课程 / 讲义 / 真题等资料库
cur.execute("""
    CREATE TABLE IF NOT EXISTS materials (
        id         INTEGER PRIMARY KEY AUTOINCREMENT,
        title      TEXT NOT NULL,
        category   TEXT NOT NULL DEFAULT '行测',
        mtype      TEXT DEFAULT '书籍',
        location   TEXT DEFAULT '',
        status     TEXT NOT NULL DEFAULT '未开始',
        rating     INTEGER DEFAULT 0,
        notes      TEXT DEFAULT '',
        created_at TEXT NOT NULL,
        updated_at TEXT NOT NULL
    )
""")

# 待办事项表：支持「今日三件事」标记
cur.execute("""
    CREATE TABLE IF NOT EXISTS todos (
        id         INTEGER PRIMARY KEY AUTOINCREMENT,
        title      TEXT NOT NULL,
        category   TEXT NOT NULL DEFAULT '其他',
        priority   TEXT NOT NULL DEFAULT '中',
        due_date   TEXT DEFAULT '',
        done       INTEGER NOT NULL DEFAULT 0,
        is_top3    INTEGER NOT NULL DEFAULT 0,
        created_at TEXT NOT NULL,
        updated_at TEXT NOT NULL
    )
""")

# 考试日程表：笔试 / 面试等关键时间节点
cur.execute("""
    CREATE TABLE IF NOT EXISTS exams (
        id                  INTEGER PRIMARY KEY AUTOINCREMENT,
        name                TEXT NOT NULL,
        exam_date           TEXT NOT NULL,
        location            TEXT DEFAULT '',
        registration_deadline TEXT DEFAULT '',
        notes               TEXT DEFAULT '',
        created_at          TEXT NOT NULL,
        updated_at          TEXT NOT NULL
    )
""")

# 操作日志表：记录关键数据变更，便于追溯
cur.execute("""
    CREATE TABLE IF NOT EXISTS op_logs (
        id         INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
            action     TEXT NOT NULL,
            target     TEXT NOT NULL,
            detail     TEXT DEFAULT '',
            created_at TEXT NOT NULL
        )
    """)

    # 元信息表：存放 schema 版本等键值
    cur.execute("""
        CREATE TABLE IF NOT EXISTS meta (
            key   TEXT PRIMARY KEY,
            value TEXT NOT NULL
        )
    """)
    cur.execute(
        "INSERT OR IGNORE INTO meta (key, value) VALUES ('schema_version', ?)",
        (str(SCHEMA_VERSION),),
    )
    conn.commit()

# -- 通用辅助 ------------------------------------------------------------
@staticmethod
def _now():
    return datetime.now().strftime("%Y-%m-%d %H:%M:%S")

@staticmethod
def _today():
    return datetime.now().strftime("%Y-%m-%d")

def _fetch_all(self, sql, params=()):
    cur = self.connect().cursor()
    cur.execute(sql, params)
    return [dict(row) for row in cur.fetchall()]

def _fetch_one(self, sql, params=()):
    cur = self.connect().cursor()
    cur.execute(sql, params)
    row = cur.fetchone()
    return dict(row) if row else None

def _execute(self, sql, params=()):
    conn = self.connect()
    cur = conn.cursor()
    try:
        cur.execute(sql, params)
        conn.commit()
        return cur.lastrowid
    except Exception:
        conn.rollback()
        raise

def log(self, action, target, detail=""):
    """写入一条操作日志（内部调用，不对外暴露为 REST 资源）。"""
    try:
        self._execute(
            "INSERT INTO op_logs (action, target, detail, created_at)"
            " VALUES (?, ?, ?, ?)",
            (action, target, detail, self._now()),
        )
    except Exception:
        # 日志写入失败不应影响主业务流程
        pass
```

```
# ---------------------------------------------------------------------------
# 用户账号
# ---------------------------------------------------------------------------

def _hash_password(password, salt):
    """PBKDF2-HMAC-SHA256 密码哈希，20 万轮迭代。"""
    dk = hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"),
                             salt.encode("utf-8"), 200000)
    return dk.hex()


class UserStore:
    """用户账号管理：首次运行创建默认账号，后续支持改密。"""

    DEFAULT_USERNAME = "admin"
    DEFAULT_PASSWORD = "123456"

    def __init__(self, db):
        self.db = db
        self._ensure_default_user()

    def _ensure_default_user(self):
        """数据库为空时创建默认账号 admin / 123456（首次登录后请立即修改）。"""
        existing = self.db._fetch_one("SELECT id FROM users LIMIT 1")
        if existing is None:
            salt = os.urandom(16).hex()
            self.db._execute(
                "INSERT INTO users (username, password_hash, salt, created_at)"
                " VALUES (?, ?, ?, ?)",
                (self.DEFAULT_USERNAME,
                 _hash_password(self.DEFAULT_PASSWORD, salt),
                 salt, self.db._now()),
            )

    def verify(self, username, password):
        """校验用户名密码，成功返回用户信息字典，失败返回 None。"""
        row = self.db._fetch_one(
            "SELECT * FROM users WHERE username = ?", (username,))
        if not row:
            return None
        if _hash_password(password, row["salt"]) == row["password_hash"]:
            return {"id": row["id"], "username": row["username"],
                    "created_at": row["created_at"]}
        return None

    def change_password(self, username, old_password, new_password):
        """修改密码：需先验证旧密码；新密码长度至少 6 位。"""
        if not new_password or len(new_password) < 6:
            return False, "新密码长度至少 6 位"
        user = self.verify(username, old_password)
        if not user:
            return False, "原密码不正确"
        salt = os.urandom(16).hex()
        self.db._execute(
            "UPDATE users SET password_hash = ?, salt = ? WHERE username = ?",
            (_hash_password(new_password, salt), salt, username),
        )
        self.db.log("修改密码", "users", username)
        return True, "密码修改成功"
```

```
# ---------------------------------------------------------------------------
# 求职投递
# ---------------------------------------------------------------------------

class JobStore:
    """求职投递记录的增删改查。"""

    def __init__(self, db):
        self.db = db

    def _validate(self, data):
        if not (data.get("company") or "").strip():
            return "公司名称不能为空"
        if not (data.get("position") or "").strip():
            return "投递岗位不能为空"
        if data.get("stage") and data["stage"] not in JOB_STAGES:
            return "投递阶段不合法"
        if data.get("priority") and data["priority"] not in TODO_PRIORITIES:
            return "优先级不合法"
        return None

    def create(self, data):
        err = self._validate(data)
        if err:
            return None, err
        now = self.db._now()
        rid = self.db._execute(
            """INSERT INTO job_applications
               (company, position, city, stage, source, applied_date, deadline,
                salary_range, priority, notes, created_at, updated_at)
               VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)""",
            (data.get("company", "").strip(), data.get("position", "").strip(),
             data.get("city", ""), data.get("stage", "待投递"),
             data.get("source", ""), data.get("applied_date", ""),
             data.get("deadline", ""), data.get("salary_range", ""),
             data.get("priority", "中"), data.get("notes", ""), now, now),
        )
        self.db.log("新增", "job_applications",
                    "%s %s" % (data.get("company"), data.get("position")))
        return self.get(rid), None

    def get(self, rid):
        return self.db._fetch_one(
            "SELECT * FROM job_applications WHERE id = ?", (rid,))

    def list(self, stage=None, keyword=None, order="updated_at DESC"):
        """按阶段 / 关键字筛选投递记录。"""
        sql = "SELECT * FROM job_applications WHERE 1=1"
        params = []
        if stage:
            sql += " AND stage = ?"
            params.append(stage)
        if keyword:
            sql += " AND (company LIKE ? OR position LIKE ? OR city LIKE ?)"
            kw = "%%%s%%" % keyword
            params += [kw, kw, kw]
        sql += " ORDER BY " + order
        return self.db._fetch_all(sql, params)

    def update(self, rid, data):
        old = self.get(rid)
```

```
        if not old:
            return None, "记录不存在"
        merged = dict(old)
        merged.update({k: v for k, v in data.items()
                       if k in ("company", "position", "city", "stage", "source",
                                "applied_date", "deadline", "salary_range",
                                "priority", "notes")})
        err = self._validate(merged)
        if err:
            return None, err
        self.db._execute(
            """UPDATE job_applications SET company=?, position=?, city=?, stage=?,
               source=?, applied_date=?, deadline=?, salary_range=?, priority=?,
               notes=?, updated_at=? WHERE id=?""",
            (merged["company"], merged["position"], merged["city"],
             merged["stage"], merged["source"], merged["applied_date"],
             merged["deadline"], merged["salary_range"], merged["priority"],
             merged["notes"], self.db._now(), rid),
        )
        self.db.log("更新", "job_applications",
                    "%s %s -> %s" % (merged["company"], merged["position"],
                                     merged["stage"]))
        return self.get(rid), None

    def delete(self, rid):
        old = self.get(rid)
        if not old:
            return False, "记录不存在"
        self.db._execute("DELETE FROM job_applications WHERE id = ?", (rid,))
        self.db.log("删除", "job_applications",
                    "%s %s" % (old["company"], old["position"]))
        return True, None

    def count_by_stage(self):
        """各阶段投递数量统计，用于看板与漏斗图。"""
        rows = self.db._fetch_all(
            "SELECT stage, COUNT(*) AS c FROM job_applications GROUP BY stage")
        result = {s: 0 for s in JOB_STAGES}
        for r in rows:
            if r["stage"] in result:
                result[r["stage"]] = r["c"]
        return result

    def upcoming_deadlines(self, days=14):
        """未来 N 天内截止的投递（按截止日期升序）。"""
        today = self.db._today()
        rows = self.db._fetch_all(
            """SELECT * FROM job_applications
               WHERE deadline != '' AND deadline >= ?
                 AND stage NOT IN ('Offer', '已拒绝')
               ORDER BY deadline ASC""", (today,))
        return rows[:50]


# ---------------------------------------------------------------------------
# 笔试练习
# ---------------------------------------------------------------------------

class PracticeStore:
    """笔试练习记录的增删改查与正确率聚合。"""

    def __init__(self, db):
```

```
    self.db = db

def _validate(self, data):
    if data.get("module") not in PRACTICE_MODULES:
        return "练习模块不合法"
    try:
        total = int(data.get("question_count", 0))
        correct = int(data.get("correct_count", 0))
    except (TypeError, ValueError):
        return "题量与答对数必须为整数"
    if total <= 0:
        return "练习题量必须大于 0"
    if not (0 <= correct <= total):
        return "答对数应在 0 到题量之间"
    return None

def create(self, data):
    err = self._validate(data)
    if err:
        return None, err
    rid = self.db._execute(
        """INSERT INTO practice_sessions
           (module, question_count, correct_count, duration_min,
            practiced_at, notes, created_at)
           VALUES (?, ?, ?, ?, ?, ?, ?)""",
        (data["module"], int(data["question_count"]),
         int(data["correct_count"]),
         float(data.get("duration_min") or 0),
         data.get("practiced_at") or self.db._today(),
         data.get("notes", ""), self.db._now()),
    )
    self.db.log("新增", "practice_sessions",
                "%s %s题" % (data["module"], data["question_count"]))
    return self.get(rid), None

def get(self, rid):
    return self.db._fetch_one(
        "SELECT * FROM practice_sessions WHERE id = ?", (rid,))

def list(self, module=None, limit=200):
    sql = "SELECT * FROM practice_sessions WHERE 1=1"
    params = []
    if module:
        sql += " AND module = ?"
        params.append(module)
    sql += " ORDER BY practiced_at DESC, id DESC LIMIT ?"
    params.append(limit)
    return self.db._fetch_all(sql, params)

def update(self, rid, data):
    old = self.get(rid)
    if not old:
        return None, "记录不存在"
    merged = dict(old)
    merged.update({k: v for k, v in data.items()
                   if k in ("module", "question_count", "correct_count",
                            "duration_min", "practiced_at", "notes")})
    err = self._validate(merged)
    if err:
        return None, err
    self.db._execute(
        """UPDATE practice_sessions SET module=?, question_count=?,
```

```
                correct_count=?, duration_min=?, practiced_at=?, notes=?
                WHERE id=?""",
            (merged["module"], int(merged["question_count"]),
             int(merged["correct_count"]), float(merged["duration_min"] or 0),
             merged["practiced_at"], merged["notes"], rid),
        )
        return self.get(rid), None

    def delete(self, rid):
        if not self.get(rid):
            return False, "记录不存在"
        self.db._execute("DELETE FROM practice_sessions WHERE id = ?", (rid,))
        self.db.log("删除", "practice_sessions", "id=%s" % rid)
        return True, None

    def accuracy_by_module(self):
        """各模块累计正确率：答对总数 / 练习总数。"""
        rows = self.db._fetch_all(
            """SELECT module, SUM(question_count) AS total,
                      SUM(correct_count) AS correct
               FROM practice_sessions GROUP BY module""")
        result = {}
        for r in rows:
            total = r["total"] or 0
            result[r["module"]] = {
                "total": total,
                "correct": r["correct"] or 0,
                "accuracy": round((r["correct"] or 0) / total, 4)
                if total else 0,
            }
        return result

    def daily_trend(self, days=30):
        """最近 N 天每日练习量与正确率趋势。"""
        rows = self.db._fetch_all(
            """SELECT practiced_at AS d, SUM(question_count) AS total,
                      SUM(correct_count) AS correct,
                      SUM(duration_min) AS minutes
               FROM practice_sessions
               GROUP BY practiced_at ORDER BY practiced_at DESC LIMIT ?""",
            (days,))
        trend = []
        for r in reversed(rows):
            total = r["total"] or 0
            trend.append({
                "date": r["d"], "total": total,
                "correct": r["correct"] or 0,
                "minutes": round(r["minutes"] or 0, 1),
                "accuracy": round((r["correct"] or 0) / total, 4)
                if total else 0,
            })
        return trend


# ---------------------------------------------------------------------------
# 备考资料
# ---------------------------------------------------------------------------

class MaterialStore:
    """备考资料库的增删改查。"""

    def __init__(self, db):
```

```
    self.db = db

def _validate(self, data):
    if not (data.get("title") or "").strip():
        return "资料标题不能为空"
    if data.get("category") and data["category"] not in MATERIAL_CATEGORIES:
        return "资料分类不合法"
    if data.get("status") and data["status"] not in MATERIAL_STATUS:
        return "学习状态不合法"
    return None

def create(self, data):
    err = self._validate(data)
    if err:
        return None, err
    now = self.db._now()
    rid = self.db._execute(
        """INSERT INTO materials
           (title, category, mtype, location, status, rating, notes,
            created_at, updated_at)
           VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
        (data["title"].strip(), data.get("category", "行测"),
         data.get("mtype", "书籍"), data.get("location", ""),
         data.get("status", "未开始"), int(data.get("rating") or 0),
         data.get("notes", ""), now, now),
    )
    return self.get(rid), None

def get(self, rid):
    return self.db._fetch_one(
        "SELECT * FROM materials WHERE id = ?", (rid,))

def list(self, category=None, status=None, keyword=None):
    sql = "SELECT * FROM materials WHERE 1=1"
    params = []
    if category:
        sql += " AND category = ?"
        params.append(category)
    if status:
        sql += " AND status = ?"
        params.append(status)
    if keyword:
        sql += " AND (title LIKE ? OR notes LIKE ?)"
        kw = "%%%s%%" % keyword
        params += [kw, kw]
    sql += " ORDER BY updated_at DESC"
    return self.db._fetch_all(sql, params)

def update(self, rid, data):
    old = self.get(rid)
    if not old:
        return None, "记录不存在"
    merged = dict(old)
    merged.update({k: v for k, v in data.items()
                   if k in ("title", "category", "mtype", "location",
                            "status", "rating", "notes")})
    err = self._validate(merged)
    if err:
        return None, err
    self.db._execute(
        """UPDATE materials SET title=?, category=?, mtype=?, location=?,
           status=?, rating=?, notes=?, updated_at=? WHERE id=?""",
```

```
            (merged["title"], merged["category"], merged["mtype"],
             merged["location"], merged["status"],
             int(merged["rating"] or 0), merged["notes"],
             self.db._now(), rid),
        )
        return self.get(rid), None

    def delete(self, rid):
        if not self.get(rid):
            return False, "记录不存在"
        self.db._execute("DELETE FROM materials WHERE id = ?", (rid,))
        return True, None

    def count_by_status(self):
        rows = self.db._fetch_all(
            "SELECT status, COUNT(*) AS c FROM materials GROUP BY status")
        result = {s: 0 for s in MATERIAL_STATUS}
        for r in rows:
            if r["status"] in result:
                result[r["status"]] = r["c"]
        return result


# ---------------------------------------------------------------------------
# 待办事项
# ---------------------------------------------------------------------------

class TodoStore:
    """待办事项的增删改查，支持「今日三件事」标记与完成切换。"""

    def __init__(self, db):
        self.db = db

    def _validate(self, data):
        if not (data.get("title") or "").strip():
            return "待办标题不能为空"
        if data.get("category") and data["category"] not in TODO_CATEGORIES:
            return "待办分类不合法"
        if data.get("priority") and data["priority"] not in TODO_PRIORITIES:
            return "优先级不合法"
        return None

    def create(self, data):
        err = self._validate(data)
        if err:
            return None, err
        now = self.db._now()
        rid = self.db._execute(
            """INSERT INTO todos
               (title, category, priority, due_date, done, is_top3,
                created_at, updated_at)
               VALUES (?, ?, ?, ?, 0, ?, ?, ?)""",
            (data["title"].strip(), data.get("category", "其他"),
             data.get("priority", "中"), data.get("due_date", ""),
             1 if data.get("is_top3") else 0, now, now),
        )
        return self.get(rid), None

    def get(self, rid):
        return self.db._fetch_one("SELECT * FROM todos WHERE id = ?", (rid,))

    def list(self, done=None, category=None):
```

```
    sql = "SELECT * FROM todos WHERE 1=1"
    params = []
    if done is not None:
        sql += " AND done = ?"
        params.append(1 if done else 0)
    if category:
        sql += " AND category = ?"
        params.append(category)
    # 未完成的排前面：今日三件事优先，其次按优先级与截止日期
    sql += (" ORDER BY done ASC, is_top3 DESC,"
            " CASE priority WHEN '高' THEN 0 WHEN '中' THEN 1 ELSE 2 END,"
            " due_date ASC, updated_at DESC")
    return self.db._fetch_all(sql, params)

def update(self, rid, data):
    old = self.get(rid)
    if not old:
        return None, "记录不存在"
    merged = dict(old)
    merged.update({k: v for k, v in data.items()
                   if k in ("title", "category", "priority", "due_date",
                            "done", "is_top3")})
    err = self._validate(merged)
    if err:
        return None, err
    self.db._execute(
        """UPDATE todos SET title=?, category=?, priority=?, due_date=?,
           done=?, is_top3=?, updated_at=? WHERE id=?""",
        (merged["title"], merged["category"], merged["priority"],
         merged["due_date"], 1 if merged["done"] else 0,
         1 if merged["is_top3"] else 0, self.db._now(), rid),
    )
    return self.get(rid), None

def toggle_done(self, rid):
    old = self.get(rid)
    if not old:
        return None, "记录不存在"
    new_done = 0 if old["done"] else 1
    self.db._execute(
        "UPDATE todos SET done = ?, updated_at = ? WHERE id = ?",
        (new_done, self.db._now(), rid),
    )
    return self.get(rid), None

def delete(self, rid):
    if not self.get(rid):
        return False, "记录不存在"
    self.db._execute("DELETE FROM todos WHERE id = ?", (rid,))
    return True, None

def top3(self):
    """今日三件事：标记为 is_top3 且未完成的事项。"""
    return self.db._fetch_all(
        "SELECT * FROM todos WHERE is_top3 = 1 AND done = 0"
        " ORDER BY updated_at DESC LIMIT 3")

def overdue(self):
    """已过期未完成的事项（截止日期早于今天）。"""
    today = self.db._today()
    return self.db._fetch_all(
        "SELECT * FROM todos WHERE done = 0 AND due_date != ''"
```

```
            " AND due_date < ? ORDER BY due_date ASC", (today,))


# ---------------------------------------------------------------------------
# 考试日程
# ---------------------------------------------------------------------------

class ExamStore:
    """笔试 / 面试等关键时间节点的增删改查。"""

    def __init__(self, db):
        self.db = db

    def _validate(self, data):
        if not (data.get("name") or "").strip():
            return "考试名称不能为空"
        if not (data.get("exam_date") or "").strip():
            return "考试日期不能为空"
        return None

    def create(self, data):
        err = self._validate(data)
        if err:
            return None, err
        now = self.db._now()
        rid = self.db._execute(
            """INSERT INTO exams
               (name, exam_date, location, registration_deadline, notes,
                created_at, updated_at)
               VALUES (?, ?, ?, ?, ?, ?, ?)""",
            (data["name"].strip(), data["exam_date"],
             data.get("location", ""),
             data.get("registration_deadline", ""),
             data.get("notes", ""), now, now),
        )
        return self.get(rid), None

    def get(self, rid):
        return self.db._fetch_one("SELECT * FROM exams WHERE id = ?", (rid,))

    def list(self, upcoming_only=False):
        sql = "SELECT * FROM exams"
        params = []
        if upcoming_only:
            sql += " WHERE exam_date >= ?"
            params.append(self.db._today())
        sql += " ORDER BY exam_date ASC"
        rows = self.db._fetch_all(sql, params)
        # 附加「距今天数」便于前端展示倒计时
        today = datetime.now().date()
        for r in rows:
            try:
                d = datetime.strptime(r["exam_date"][:10], "%Y-%m-%d").date()
                r["days_left"] = (d - today).days
            except (ValueError, TypeError):
                r["days_left"] = None
        return rows

    def update(self, rid, data):
        old = self.get(rid)
        if not old:
            return None, "记录不存在"
```

```
        merged = dict(old)
        merged.update({k: v for k, v in data.items()
                       if k in ("name", "exam_date", "location",
                                "registration_deadline", "notes")})
        err = self._validate(merged)
        if err:
            return None, err
        self.db._execute(
            """UPDATE exams SET name=?, exam_date=?, location=?,
               registration_deadline=?, notes=?, updated_at=? WHERE id=?""",
            (merged["name"], merged["exam_date"], merged["location"],
             merged["registration_deadline"], merged["notes"],
             self.db._now(), rid),
        )
        return self.get(rid), None

    def delete(self, rid):
        if not self.get(rid):
            return False, "记录不存在"
        self.db._execute("DELETE FROM exams WHERE id = ?", (rid,))
        return True, None


# ---------------------------------------------------------------------------
# 导出为 JSON（数据备份）
# ---------------------------------------------------------------------------

def export_all(db):
    """导出全部业务数据为字典，用于备份下载。"""
    tables = ["job_applications", "practice_sessions", "materials",
              "todos", "exams"]
    data = {"exported_at": db._now(), "tables": {}}
    for t in tables:
        data["tables"][t] = db._fetch_all("SELECT * FROM %s" % t)
    return data


def import_all(db, data):
    """从 export_all 的备份数据恢复（先清空再写入）。"""
    tables = ["job_applications", "practice_sessions", "materials",
              "todos", "exams"]
    payload = data.get("tables", {})
    conn = db.connect()
    cur = conn.cursor()
    try:
        for t in tables:
            rows = payload.get(t, [])
            if not rows:
                continue
            cur.execute("DELETE FROM %s" % t)
            cols = [c for c in rows[0].keys() if c != "id"]
            placeholders = ", ".join(["?"] * len(cols))
            for r in rows:
                cur.execute(
                    "INSERT INTO %s (%s) VALUES (%s)"
                    % (t, ", ".join(cols), placeholders),
                    [r[c] for c in cols],
                )
        conn.commit()
        db.log("导入备份", "database",
               "共 %d 张表" % sum(1 for t in tables if payload.get(t)))
        return True, "导入成功"
```

```
    except Exception as e:
        conn.rollback()
        return False, "导入失败：%s" % str(e)[:100]
```

——— backend/stats.py ———

```
# -*- coding: utf-8 -*-
"""
求职备考管理软件 V1.0
统计分析模块：为「数据总览」看板提供各类聚合指标。

本模块只做只读聚合计算，不修改任何业务数据。所有指标均由
models 层的基础查询组合而成，便于单元测试与复用。
"""

from datetime import datetime, timedelta


def _parse_date(s):
    """解析 YYYY-MM-DD，失败返回 None。"""
    try:
        return datetime.strptime((s or "")[:10], "%Y-%m-%d").date()
    except (ValueError, TypeError):
        return None


def overview(db, jobs, practice, materials, todos, exams):
    """首页总览看板的一站式数据包。

    返回字典包含：
      jobs:      求职投递总数 / 各阶段数量 / Offer 率 / 近 14 天新增
      practice:  累计练习题量 / 累计答对 / 总体正确率 / 累计时长 / 近 7 天题量
      materials: 资料总数 / 各状态数量 / 完成率
      todos:     待办总数 / 未完成数 / 逾期数 / 今日三件事完成情况
      exams:     临近考试（30 天内）列表
    """
    today = datetime.now().date()

    # ---- 求职投递 ----
    stage_counts = jobs.count_by_stage()
    total_jobs = sum(stage_counts.values())
    offer_count = stage_counts.get("Offer", 0)
    # Offer 率 = Offer 数 / 已出结果数（Offer + 已拒绝），避免分母为 0
    decided = offer_count + stage_counts.get("已拒绝", 0)
    offer_rate = round(offer_count / decided, 4) if decided else 0
    active_jobs = total_jobs - offer_count - stage_counts.get("已拒绝", 0)

    # ---- 笔试练习 ----
    acc_modules = practice.accuracy_by_module()
    total_q = sum(v["total"] for v in acc_modules.values())
    total_c = sum(v["correct"] for v in acc_modules.values())
    overall_acc = round(total_c / total_q, 4) if total_q else 0
    trend = practice.daily_trend(days=7)
    week_q = sum(d["total"] for d in trend)
    week_minutes = round(sum(d["minutes"] for d in trend), 1)
    # 正确率最低的模块 → 薄弱环节提示
    weakest = None
    if acc_modules:
        weakest = min(acc_modules.items(), key=lambda kv: kv[1]["accuracy"])[0]

    # ---- 备考资料 ----
    mat_status = materials.count_by_status()
    total_mat = sum(mat_status.values())
    mat_done_rate = (round(mat_status.get("已完成", 0) / total_mat, 4)
```

```
                    if total_mat else 0)

    # ---- 待办事项 ----
    all_todos = todos.list()
    undone = [t for t in all_todos if not t["done"]]
    overdue_list = todos.overdue()
    top3_list = todos.top3()

    # ---- 临近考试 ----
    upcoming_exams = []
    for e in exams.list(upcoming_only=True):
        if e["days_left"] is not None and e["days_left"] <= 30:
            upcoming_exams.append(e)

    return {
        "jobs": {
            "total": total_jobs,
            "active": active_jobs,
            "by_stage": stage_counts,
            "offer_rate": offer_rate,
            "upcoming_deadlines": jobs.upcoming_deadlines(days=14),
        },
        "practice": {
            "total_questions": total_q,
            "total_correct": total_c,
            "overall_accuracy": overall_acc,
            "week_questions": week_q,
            "week_minutes": week_minutes,
            "by_module": acc_modules,
            "weakest_module": weakest,
            "daily_trend": trend,
        },
        "materials": {
            "total": total_mat,
            "by_status": mat_status,
            "done_rate": mat_done_rate,
        },
        "todos": {
            "total": len(all_todos),
            "undone": len(undone),
            "overdue": len(overdue_list),
            "overdue_list": overdue_list[:5],
            "top3": top3_list,
        },
        "exams": {
            "upcoming": upcoming_exams,
        },
        "generated_at": datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
    }


def job_funnel(jobs):
    """求职漏斗：按阶段顺序返回各阶段数量与转化率。

    转化率口径：本阶段数量 / 上一阶段数量（首阶段为 100%）。
    """
    counts = jobs.count_by_stage()
    order = ["待投递", "已投递", "笔试", "面试", "Offer"]
    funnel = []
    prev = None
    for stage in order:
```

```
        c = counts.get(stage, 0)
        rate = 1.0 if prev is None else (round(c / prev, 4) if prev else 0)
        funnel.append({"stage": stage, "count": c, "rate": rate})
        prev = c
    funnel.append({"stage": "已拒绝", "count": counts.get("已拒绝", 0),
                   "rate": None})
    return funnel


def practice_weekly_report(practice, weeks=8):
    """按自然周聚合练习数据，用于周报式的进步曲线。

    返回最近 N 周每周的：题量 / 答对 / 正确率 / 时长（分钟）。
    """
    sessions = practice.list(limit=10000)
    buckets = {}
    for s in sessions:
        d = _parse_date(s.get("practiced_at"))
        if not d:
            continue
        # 以周一为一周的开始
        monday = d - timedelta(days=d.weekday())
        key = monday.strftime("%Y-%m-%d")
        b = buckets.setdefault(key, {"week_start": key, "total": 0,
                                     "correct": 0, "minutes": 0.0})
        b["total"] += s["question_count"] or 0
        b["correct"] += s["correct_count"] or 0
        b["minutes"] += s["duration_min"] or 0
    weeks_sorted = sorted(buckets.values(),
                          key=lambda x: x["week_start"], reverse=True)[:weeks]
    for w in weeks_sorted:
        total = w["total"]
        w["accuracy"] = round(w["correct"] / total, 4) if total else 0
        w["minutes"] = round(w["minutes"], 1)
    return list(reversed(weeks_sorted))


def module_radar(practice):
    """各模块能力雷达：题量与正确率，用于定位薄弱模块。"""
    acc = practice.accuracy_by_module()
    radar = []
    for module in ["图形推理", "逻辑判断", "数量分析",
                   "资料分析", "言语理解", "常识判断"]:
        v = acc.get(module, {"total": 0, "correct": 0, "accuracy": 0})
        radar.append({
            "module": module,
            "total": v["total"],
            "accuracy": v["accuracy"],
            # 熟练度综合分：正确率 * min(1, 题量/200)，题量不足则打折
            "proficiency": round(v["accuracy"] * min(1.0, v["total"] / 200), 4),
        })
    return radar


def material_progress_by_category(db, materials):
    """按分类统计资料完成情况。"""
    rows = db._fetch_all(
        """SELECT category,
                  COUNT(*) AS total,
                  SUM(CASE WHEN status='已完成' THEN 1 ELSE 0 END) AS done
           FROM materials GROUP BY category""")
```

```
    result = []
    for r in rows:
        total = r["total"] or 0
        done = r["done"] or 0
        result.append({
            "category": r["category"], "total": total, "done": done,
            "rate": round(done / total, 4) if total else 0,
        })
    return sorted(result, key=lambda x: x["total"], reverse=True)


def todo_completion_trend(db, days=14):
    """最近 N 天待办完成趋势（按 updated_at 且 done=1 聚合）。"""
    rows = db._fetch_all(
        """SELECT substr(updated_at, 1, 10) AS d, COUNT(*) AS c
           FROM todos WHERE done = 1 GROUP BY d
           ORDER BY d DESC LIMIT ?""", (days,))
    trend = [{"date": r["d"], "done_count": r["c"]} for r in reversed(rows)]
    return trend
```

——— backend/seed.py ———

```
# -*- coding: utf-8 -*-
"""
求职备考管理软件 V1.0
演示数据播种：写入一组贴近真实使用场景的示例数据。

用途：
  1. 新用户首次运行时一键体验全部功能
  2. 为使用手册截图提供统一的演示数据
  3. 自动化测试的基准数据集

注意：演示数据中的公司 / 人名均为虚构，仅用于功能演示。
调用时若 clear_first=True，会先清空五张业务表再写入。
"""

from datetime import datetime, timedelta

from models import (ExamStore, JobStore, MaterialStore, PracticeStore,
                    TodoStore)


def _days_ago(n):
    return (datetime.now() - timedelta(days=n)).strftime("%Y-%m-%d")


def _days_later(n):
    return (datetime.now() + timedelta(days=n)).strftime("%Y-%m-%d")


# ---------------------------------------------------------------------------
# 各实体的演示数据
# ---------------------------------------------------------------------------

DEMO_JOBS = [
    {"company": "星云科技", "position": "产品经理", "city": "杭州",
     "stage": "面试", "source": "校园招聘", "applied_date": _days_ago(20),
     "deadline": _days_later(5), "salary_range": "18-25K", "priority": "高",
     "notes": "一面已通过，等待二面通知，提前准备产品分析案例。"},
    {"company": "蓝桥软件", "position": "产品助理", "city": "合肥",
     "stage": "笔试", "source": "招聘网站", "applied_date": _days_ago(12),
     "deadline": _days_later(3), "salary_range": "12-16K", "priority": "高",
     "notes": "笔试为行测+产品案例分析，重点复习逻辑判断模块。"},
```

```
    {"company": "恒信证券", "position": "金融科技岗", "city": "上海",
     "stage": "已投递", "source": "公司官网", "applied_date": _days_ago(6),
     "deadline": _days_later(10), "salary_range": "20-28K", "priority": "中",
     "notes": "网申已提交，留意短信与邮件通知。"},
    {"company": "云启数据", "position": "数据产品经理", "city": "北京",
     "stage": "已投递", "source": "内推", "applied_date": _days_ago(4),
     "deadline": "", "salary_range": "22-30K", "priority": "高",
     "notes": "学长内推，已确认简历进入筛选环节。"},
    {"company": "晨光文具", "position": "运营专员", "city": "上海",
     "stage": "Offer", "source": "校园招聘", "applied_date": _days_ago(45),
     "deadline": "", "salary_range": "10-14K", "priority": "低",
     "notes": "已收到 Offer，作为保底选项，答复截止下周。"},
    {"company": "远方物流", "position": "管培生", "city": "合肥",
     "stage": "已拒绝", "source": "招聘网站", "applied_date": _days_ago(50),
     "deadline": "", "salary_range": "8-12K", "priority": "低",
     "notes": "一面未通过，反思：对行业理解准备不足。"},
    {"company": "智行汽车", "position": "产品经理", "city": "合肥",
     "stage": "待投递", "source": "校园招聘", "applied_date": "",
     "deadline": _days_later(7), "salary_range": "15-20K", "priority": "中",
     "notes": "网申截止临近，需针对性优化项目经历描述。"},
    {"company": "安信银行", "position": "金融科技管培生", "city": "杭州",
     "stage": "待投递", "source": "公司官网", "applied_date": "",
     "deadline": _days_later(12), "salary_range": "18-24K", "priority": "中",
     "notes": ""},
]

DEMO_PRACTICE = [
    # (模块， 题量， 答对， 时长分钟， N天前， 备注)
    ("图形推理", 20, 15, 25, 13, "折叠展开类错误较多"),
    ("逻辑判断", 20, 14, 30, 13, "加强削弱题型需总结"),
    ("数量分析", 15, 9, 35, 12, "工程问题公式不熟"),
    ("图形推理", 20, 17, 22, 11, "有进步"),
    ("资料分析", 20, 16, 30, 11, "速算技巧见效"),
    ("言语理解", 20, 15, 25, 10, ""),
    ("逻辑判断", 25, 19, 35, 9, "翻译推理基本掌握"),
    ("数量分析", 20, 13, 40, 8, "行程问题仍是短板"),
    ("图形推理", 25, 21, 28, 7, ""),
    ("常识判断", 20, 12, 20, 6, "法律常识欠缺"),
    ("资料分析", 25, 21, 32, 5, "正确率稳定在 80%+"),
    ("逻辑判断", 20, 17, 30, 4, ""),
    ("数量分析", 20, 15, 35, 3, "错题本回顾有效"),
    ("图形推理", 20, 18, 24, 2, ""),
    ("言语理解", 25, 20, 30, 1, "中心理解题全对"),
    ("逻辑判断", 20, 18, 28, 0, "今日练习"),
]

DEMO_MATERIALS = [
    {"title": "行测模块宝典（2026 版）", "category": "行测",
     "mtype": "书籍", "location": "书架第一层", "status": "学习中",
     "rating": 5, "notes": "按模块刷题，每天一章。"},
    {"title": "粉笔 980 系统班讲义", "category": "行测",
     "mtype": "讲义", "location": "D:/备考/讲义", "status": "学习中",
     "rating": 4, "notes": "配合视频课程使用。"},
    {"title": "近五年国考真题套卷", "category": "行测",
     "mtype": "真题", "location": "D:/备考/真题", "status": "未开始",
     "rating": 5, "notes": "考前一个月开始模考。"},
    {"title": "申论范文 100 篇", "category": "申论",
     "mtype": "书籍", "location": "书架第二层", "status": "未开始",
     "rating": 4, "notes": ""},
    {"title": "半月谈时政热点合集", "category": "申论",
     "mtype": "课程", "location": "在线课程", "status": "学习中",
```

```
     "rating": 4, "notes": "每天积累 3 个金句。"},
    {"title": "产品经理面试题库", "category": "面试",
     "mtype": "文档", "location": "D:/求职/面试", "status": "已完成",
     "rating": 5, "notes": "高频题已背熟。"},
    {"title": "简历优化指南", "category": "简历",
     "mtype": "书籍", "location": "书架第一层", "status": "已完成",
     "rating": 4, "notes": ""},
    {"title": "计量经济学（第七版）", "category": "专业课",
     "mtype": "书籍", "location": "图书馆借阅", "status": "学习中",
     "rating": 3, "notes": "结合论文选题阅读。"},
]

DEMO_TODOS = [
    {"title": "完成蓝桥软件笔试", "category": "求职",
     "priority": "高", "due_date": _days_later(3),
     "done": 0, "is_top3": 1},
    {"title": "刷逻辑判断 20 题并整理错题", "category": "备考",
     "priority": "高", "due_date": _days_later(0),
     "done": 0, "is_top3": 1},
    {"title": "优化智行汽车网申简历", "category": "求职",
     "priority": "中", "due_date": _days_later(6),
     "done": 0, "is_top3": 1},
    {"title": "背诵 3 个申论金句", "category": "备考",
     "priority": "中", "due_date": _days_later(0),
     "done": 1, "is_top3": 0},
    {"title": "整理本周笔试错题本", "category": "备考",
     "priority": "中", "due_date": _days_later(2),
     "done": 0, "is_top3": 0},
    {"title": "给导师发论文进度邮件", "category": "其他",
     "priority": "低", "due_date": _days_later(5),
     "done": 0, "is_top3": 0},
    {"title": "缴纳下季度房租", "category": "生活",
     "priority": "高", "due_date": _days_ago(1),
     "done": 0, "is_top3": 0},
    {"title": "晨跑 5 公里", "category": "生活",
     "priority": "低", "due_date": _days_ago(2),
     "done": 1, "is_top3": 0},
]

DEMO_EXAMS = [
    {"name": "蓝桥软件产品助理笔试", "exam_date": _days_later(3),
     "location": "线上", "registration_deadline": _days_ago(2),
     "notes": "行测 60 分钟 + 产品案例分析 30 分钟"},
    {"name": "国家公务员考试（笔试）", "exam_date": _days_later(54),
     "location": "合肥考区", "registration_deadline": _days_later(9),
     "notes": "行测 + 申论，报名后开始套卷模考"},
    {"name": "星云科技产品经理二面", "exam_date": _days_later(6),
     "location": "杭州市滨江区", "registration_deadline": "",
     "notes": "准备 15 分钟产品方案展示"},
]


# ---------------------------------------------------------------------------
# 播种入口
# ---------------------------------------------------------------------------

def seed_demo_data(db, clear_first=False):
    """写入演示数据，返回各表写入条数统计。"""
    jobs = JobStore(db)
    practice = PracticeStore(db)
    materials = MaterialStore(db)
```

```
    todos = TodoStore(db)
    exams = ExamStore(db)

    if clear_first:
        conn = db.connect()
        cur = conn.cursor()
        for t in ("job_applications", "practice_sessions", "materials",
                  "todos", "exams"):
            cur.execute("DELETE FROM %s" % t)
        conn.commit()

    counts = {"jobs": 0, "practice": 0, "materials": 0,
              "todos": 0, "exams": 0}

    for j in DEMO_JOBS:
        item, err = jobs.create(j)
        if item:
            counts["jobs"] += 1

    for module, total, correct, minutes, ago, notes in DEMO_PRACTICE:
        item, err = practice.create({
            "module": module, "question_count": total,
            "correct_count": correct, "duration_min": minutes,
            "practiced_at": _days_ago(ago), "notes": notes})
        if item:
            counts["practice"] += 1

    for m in DEMO_MATERIALS:
        item, err = materials.create(m)
        if item:
            counts["materials"] += 1

    for t in DEMO_TODOS:
        item, err = todos.create(t)
        if item:
            counts["todos"] += 1

    for e in DEMO_EXAMS:
        item, err = exams.create(e)
        if item:
            counts["exams"] += 1

    db.log("写入演示数据", "database", str(counts))
    return counts
```

——— backend/app.py ———

```
# -*- coding: utf-8 -*-
"""
求职备考管理软件 V1.0
Web 服务入口：Flask 应用、路由注册、会话鉴权与演示数据播种。

运行方式：
    python run.py            # 开发模式，监听 127.0.0.1:5000
    python run.py --port 8080 --init-demo   # 指定端口并写入演示数据

目录结构约定：
    jobprep/
      backend/      本目录：app.py / models.py / stats.py / seed.py
      templates/    index.html（单页应用壳）
      static/       css 与 js 前端资源
      data/        SQLite 数据库文件（首次运行时自动创建）
"""
```

```
import functools
import json
import os
import sys

from flask import (Flask, jsonify, redirect, render_template, request,
                   send_from_directory, session)

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from models import (Database, ExamStore, JobStore, MaterialStore,
                    PracticeStore, TodoStore, UserStore, export_all,
                    import_all)
import stats as stats_mod
from seed import seed_demo_data

# ---------------------------------------------------------------------------
# 应用与存储初始化
# ---------------------------------------------------------------------------

BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
DATA_DIR = os.path.join(BASE_DIR, "data")
DB_PATH = os.path.join(DATA_DIR, "jobprep.db")

APP_NAME = "求职备考管理软件"
APP_VERSION = "V1.0"


def create_app(db_path=DB_PATH):
    """应用工厂：创建 Flask 实例并装配全部路由。"""
    app = Flask(__name__,
                template_folder=os.path.join(BASE_DIR, "templates"),
                static_folder=os.path.join(BASE_DIR, "static"))
    # 会话密钥：首次运行随机生成并持久化，避免重启后会话失效
    app.secret_key = _load_or_create_secret()

    db = Database(db_path)
    users = UserStore(db)
    jobs = JobStore(db)
    practice = PracticeStore(db)
    materials = MaterialStore(db)
    todos = TodoStore(db)
    exams = ExamStore(db)

    # 挂到 app 上，便于测试与扩展访问
    app.db = db
    app.stores = {"users": users, "jobs": jobs, "practice": practice,
                  "materials": materials, "todos": todos, "exams": exams}

    # -----------------------------------------------------------------
    # 鉴权装饰器
    # -----------------------------------------------------------------
    def login_required(view):
        @functools.wraps(view)
        def wrapper(*args, **kwargs):
            if not session.get("username"):
                return jsonify({"ok": False, "error": "未登录",
                                "code": "NOT_LOGGED_IN"}), 401
            return view(*args, **kwargs)
        return wrapper
```

```
# ---------------------------------------------------------------------
# 页面路由
# ---------------------------------------------------------------------
@app.route("/")
def index():
    # 单页应用壳；前端按登录态渲染登录页或主界面
    # 截图辅助：设置环境变量 JOBPREP_AUTO_LOGIN=admin 后，可免登录
    # 直接进入主界面，仅用于制作使用手册截图，生产环境请勿开启
    if (os.environ.get("JOBPREP_AUTO_LOGIN") == "admin"
            and not session.get("username")):
        session["username"] = UserStore.DEFAULT_USERNAME
    return render_template("index.html", app_name=APP_NAME,
                           app_version=APP_VERSION)

@app.route("/health")
def health():
    return jsonify({"ok": True, "app": APP_NAME,
                    "version": APP_VERSION})

# ---------------------------------------------------------------------
# 账号接口
# ---------------------------------------------------------------------
@app.route("/api/login", methods=["POST"])
def api_login():
    data = request.get_json(force=True, silent=True) or {}
    username = (data.get("username") or "").strip()
    password = data.get("password") or ""
    user = users.verify(username, password)
    if not user:
        return jsonify({"ok": False, "error": "用户名或密码不正确"}), 400
    session["username"] = user["username"]
    session.permanent = True
    return jsonify({"ok": True, "user": user})

@app.route("/api/logout", methods=["POST"])
def api_logout():
    session.pop("username", None)
    return jsonify({"ok": True})

@app.route("/api/me")
def api_me():
    if not session.get("username"):
        return jsonify({"ok": True, "user": None})
    return jsonify({"ok": True,
                    "user": {"username": session["username"]}})

@app.route("/api/change_password", methods=["POST"])
@login_required
def api_change_password():
    data = request.get_json(force=True, silent=True) or {}
    ok, msg = users.change_password(session["username"],
                                    data.get("old_password", ""),
                                    data.get("new_password", ""))
    return jsonify({"ok": ok, "error": None if ok else msg,
                    "message": msg if ok else None})

# ---------------------------------------------------------------------
# 通用 CRUD 路由生成器
# ---------------------------------------------------------------------
def crud_routes(prefix, store, list_kwargs=()):
    """为一个 Store 批量注册 list/get/create/update/delete 路由。"""
```

```
    @app.route("/api/%s" % prefix, methods=["GET"],
               endpoint="list_" + prefix)
    @login_required
    def _list(store=store, list_kwargs=list_kwargs):
        kwargs = {}
        for key in list_kwargs:
            if key in request.args:
                kwargs[key] = request.args.get(key)
        try:
            items = store.list(**kwargs)
        except TypeError:
            items = store.list()
        return jsonify({"ok": True, "items": items})

    @app.route("/api/%s/<int:rid>" % prefix, methods=["GET"],
               endpoint="get_" + prefix)
    @login_required
    def _get(rid, store=store):
        item = store.get(rid)
        if not item:
            return jsonify({"ok": False, "error": "记录不存在"}), 404
        return jsonify({"ok": True, "item": item})

    @app.route("/api/%s" % prefix, methods=["POST"],
               endpoint="create_" + prefix)
    @login_required
    def _create(store=store):
        data = request.get_json(force=True, silent=True) or {}
        item, err = store.create(data)
        if err:
            return jsonify({"ok": False, "error": err}), 400
        return jsonify({"ok": True, "item": item})

    @app.route("/api/%s/<int:rid>" % prefix, methods=["PUT"],
               endpoint="update_" + prefix)
    @login_required
    def _update(rid, store=store):
        data = request.get_json(force=True, silent=True) or {}
        item, err = store.update(rid, data)
        if err:
            return jsonify({"ok": False, "error": err}), 400
        return jsonify({"ok": True, "item": item})

    @app.route("/api/%s/<int:rid>" % prefix, methods=["DELETE"],
               endpoint="delete_" + prefix)
    @login_required
    def _delete(rid, store=store):
        ok, err = store.delete(rid)
        if not ok:
            return jsonify({"ok": False, "error": err}), 400
        return jsonify({"ok": True})

crud_routes("jobs", jobs)
crud_routes("practice", practice)
crud_routes("materials", materials)
crud_routes("todos", todos)
crud_routes("exams", exams)

# ---------------------------------------------------------------------
# 各实体的扩展接口
```

```
# ---------------------------------------------------------------------
@app.route("/api/todos/<int:rid>/toggle", methods=["POST"])
@login_required
def api_todo_toggle(rid):
    item, err = todos.toggle_done(rid)
    if err:
        return jsonify({"ok": False, "error": err}), 400
    return jsonify({"ok": True, "item": item})

@app.route("/api/todos/top3", methods=["GET"])
@login_required
def api_todo_top3():
    return jsonify({"ok": True, "items": todos.top3()})

@app.route("/api/todos/overdue", methods=["GET"])
@login_required
def api_todo_overdue():
    return jsonify({"ok": True, "items": todos.overdue()})

@app.route("/api/jobs/by_stage", methods=["GET"])
@login_required
def api_jobs_by_stage():
    return jsonify({"ok": True, "counts": jobs.count_by_stage()})

@app.route("/api/jobs/upcoming_deadlines", methods=["GET"])
@login_required
def api_jobs_deadlines():
    days = int(request.args.get("days", 14))
    return jsonify({"ok": True,
                    "items": jobs.upcoming_deadlines(days=days)})

@app.route("/api/practice/accuracy_by_module", methods=["GET"])
@login_required
def api_practice_accuracy():
    return jsonify({"ok": True,
                    "modules": practice.accuracy_by_module()})

@app.route("/api/practice/daily_trend", methods=["GET"])
@login_required
def api_practice_trend():
    days = int(request.args.get("days", 30))
    return jsonify({"ok": True, "trend": practice.daily_trend(days=days)})

@app.route("/api/materials/by_status", methods=["GET"])
@login_required
def api_materials_status():
    return jsonify({"ok": True, "counts": materials.count_by_status()})

# ---------------------------------------------------------------------
# 统计看板接口
# ---------------------------------------------------------------------
@app.route("/api/stats/overview", methods=["GET"])
@login_required
def api_stats_overview():
    data = stats_mod.overview(db, jobs, practice, materials, todos, exams)
    return jsonify({"ok": True, "stats": data})

@app.route("/api/stats/funnel", methods=["GET"])
@login_required
def api_stats_funnel():
    return jsonify({"ok": True,
```

```
                      "funnel": stats_mod.job_funnel(jobs)})

    @app.route("/api/stats/weekly", methods=["GET"])
    @login_required
    def api_stats_weekly():
        weeks = int(request.args.get("weeks", 8))
        return jsonify({"ok": True,
                        "weeks": stats_mod.practice_weekly_report(
                            practice, weeks=weeks)})

    @app.route("/api/stats/radar", methods=["GET"])
    @login_required
    def api_stats_radar():
        return jsonify({"ok": True,
                        "radar": stats_mod.module_radar(practice)})

    @app.route("/api/stats/material_progress", methods=["GET"])
    @login_required
    def api_stats_mat_progress():
        return jsonify({"ok": True, "progress":
                        stats_mod.material_progress_by_category(db,
                                                                materials)})

    # ------------------------------------------------------------------
    # 数据备份与恢复
    # ------------------------------------------------------------------
    @app.route("/api/backup", methods=["GET"])
    @login_required
    def api_backup():
        return jsonify({"ok": True, "backup": export_all(db)})

    @app.route("/api/restore", methods=["POST"])
    @login_required
    def api_restore():
        data = request.get_json(force=True, silent=True) or {}
        ok, msg = import_all(db, data.get("backup") or {})
        return jsonify({"ok": ok, "error": None if ok else msg,
                        "message": msg if ok else None})

    @app.route("/api/seed_demo", methods=["POST"])
    @login_required
    def api_seed_demo():
        """一键写入演示数据（已存在数据时先清空）。"""
        count = seed_demo_data(db, clear_first=True)
        return jsonify({"ok": True, "message": "已写入演示数据",
                        "count": count})

    # ------------------------------------------------------------------
    # 静态资源兜底（便于直接用文件服务器预览）
    # ------------------------------------------------------------------
    @app.route("/favicon.ico")
    def favicon():
        return send_from_directory(
            os.path.join(BASE_DIR, "static"), "favicon.ico",
            mimetype="image/x-icon")

    return app


def _load_or_create_secret():
    """加载或生成 Flask 会话密钥，持久化在 data/secret.key。"""
```

```
    os.makedirs(DATA_DIR, exist_ok=True)
    key_file = os.path.join(DATA_DIR, "secret.key")
    if os.path.exists(key_file):
        with open(key_file, "rb") as f:
            return f.read().strip()
    secret = os.urandom(32)
    with open(key_file, "wb") as f:
        f.write(secret)
    try:
        os.chmod(key_file, 0o600)
    except OSError:
        pass
    return secret


def main(argv):
    """命令行入口：解析端口 / 演示数据等参数并启动服务。"""
    import argparse
    parser = argparse.ArgumentParser(
        description="%s %s" % (APP_NAME, APP_VERSION))
    parser.add_argument("--port", type=int, default=5000,
                        help="监听端口（默认 5000）")
    parser.add_argument("--host", default="127.0.0.1",
                        help="监听地址（默认 127.0.0.1）")
    parser.add_argument("--init-demo", action="store_true",
                        help="启动前写入演示数据（会先清空已有数据）")
    parser.add_argument("--debug", action="store_true",
                        help="开启 Flask 调试模式")
    args = parser.parse_args(argv)

    app = create_app()
    if args.init_demo:
        count = seed_demo_data(app.db, clear_first=True)
        print("演示数据已写入：%s" % json.dumps(count, ensure_ascii=False))
    print("%s %s 正在启动：http://%s:%d"
          % (APP_NAME, APP_VERSION, args.host, args.port))
    print("默认账号 admin / 123456（首次登录后请立即修改密码）")
    app.run(host=args.host, port=args.port, debug=args.debug)


if __name__ == "__main__":
    main(sys.argv[1:])
```

——— run.py ———

```
# -*- coding: utf-8 -*-
"""求职备考管理软件 V1.0 启动入口。"""
import os
import sys

sys.path.insert(0, os.path.join(os.path.dirname(os.path.abspath(__file__)),
                                "backend"))

from app import main

if __name__ == "__main__":
    main(sys.argv[1:])
```

——— templates/index.html ———

```
<!DOCTYPE html>
<html lang="zh-CN">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
<title>求职备考管理软件 V1.0</title>
<link rel="stylesheet" href="/static/css/style.css">
<link rel="icon" href="data:image/svg+xml,<svg xmlns='http://www.w3.org/2000/svg' viewBox='0 0
100 100'><text y='.9em' font-size='90'>📋</text></svg>">
</head>
<body>
<!-- 登录页 -->
<div id="login-page" class="login-wrap hidden">
  <div class="login-card">
    <div class="login-logo">📋</div>
    <h1 class="login-title">求职备考管理软件</h1>
    <p class="login-sub">V1.0 · 本地运行 · 数据保存在本机</p>
    <div id="login-error" class="form-error hidden"></div>
    <label class="form-label">用户名</label>
    <input id="login-username" class="form-input" type="text"
           placeholder="默认账号：admin" autocomplete="username">
    <label class="form-label">密码</label>
    <input id="login-password" class="form-input" type="password"
           placeholder="默认密码：123456" autocomplete="current-password">
    <button id="login-btn" class="btn btn-primary btn-block">登 录</button>
    <p class="login-tip">首次登录后请在「设置」中修改默认密码</p>
  </div>
</div>

<!-- 主界面 -->
<div id="app" class="hidden">
  <aside class="sidebar">
    <div class="brand">
      <span class="brand-logo">📋</span>
      <div>
        <div class="brand-name">求职备考管理软件</div>
        <div class="brand-ver">V1.0</div>
      </div>
    </div>
    <nav class="nav" id="main-nav">
      <a href="#/dashboard" class="nav-item" data-route="dashboard">
        <span class="nav-icon">🏠</span>数据总览</a>
      <a href="#/jobs" class="nav-item" data-route="jobs">
        <span class="nav-icon">💼</span>求职投递</a>
      <a href="#/practice" class="nav-item" data-route="practice">
        <span class="nav-icon">✍️</span>笔试练习</a>
      <a href="#/materials" class="nav-item" data-route="materials">
        <span class="nav-icon">📚</span>备考资料</a>
      <a href="#/todos" class="nav-item" data-route="todos">
        <span class="nav-icon">✅</span>待办清单</a>
      <a href="#/exams" class="nav-item" data-route="exams">
        <span class="nav-icon">📅</span>考试日程</a>
      <a href="#/settings" class="nav-item" data-route="settings">
        <span class="nav-icon">⚙️</span>设置</a>
    </nav>
    <div class="sidebar-foot">
      <div class="user-chip" id="user-chip">👤 <span id="username-label"></span></div>
      <button id="logout-btn" class="btn btn-ghost btn-sm">退出登录</button>
    </div>
  </aside>
  <main class="main">
    <div id="view" class="view"></div>
    <footer class="foot">求职备考管理软件 V1.0 · 数据仅保存在本地计算机</footer>
  </main>
</div>

<!-- 通用弹窗 -->
```

```
<div id="modal-mask" class="modal-mask hidden">
  <div class="modal">
    <div class="modal-head">
      <span id="modal-title" class="modal-title">标题</span>
      <button id="modal-close" class="modal-x">×</button>
    </div>
    <div id="modal-body" class="modal-body"></div>
    <div class="modal-foot">
      <button id="modal-cancel" class="btn">取消</button>
      <button id="modal-ok" class="btn btn-primary">确定</button>
    </div>
  </div>
</div>

<!-- 轻提示 -->
<div id="toast" class="toast hidden"></div>

<script src="/static/js/app.js"></script>
<script src="/static/js/view_dashboard.js"></script>
<script src="/static/js/view_jobs.js"></script>
<script src="/static/js/view_practice.js"></script>
<script src="/static/js/view_materials.js"></script>
<script src="/static/js/view_todos.js"></script>
<script src="/static/js/view_exams.js"></script>
<script src="/static/js/view_settings.js"></script>
</body>
</html>
```

——— static/css/style.css ———

```
/* 求职备考管理软件 V1.0 · 全局样式
   设计语言：浅色扁平风，主色藏青 + 活力橙点缀，圆角卡片，适配 1280px+ 桌面端 */

/* ---------- 基础与变量 ---------- */
:root {
  --primary: #1f3a5f;        /* 藏青：主色 */
  --primary-light: #2f537e;
  --accent: #f0852c;         /* 活力橙：强调 */
  --accent-light: #fdf0e2;
  --bg: #f4f6fa;             /* 页面底色 */
  --card: #ffffff;
  --text: #243447;
  --text-dim: #6b7a90;
  --border: #e3e8f0;
  --success: #22a06b;
  --warning: #e6a23c;
  --danger: #e05252;
  --radius: 10px;
  --shadow: 0 2px 10px rgba(31, 58, 95, .07);
}
* { box-sizing: border-box; margin: 0; padding: 0; }
html { font-size: 15px; }
body {
  font-family: "PingFang SC", "Microsoft YaHei", "Noto Sans SC", sans-serif;
  background: var(--bg); color: var(--text); line-height: 1.6;
}
.hidden { display: none !important; }
a { color: var(--primary-light); text-decoration: none; }

/* ---------- 登录页 ---------- */
.login-wrap {
  min-height: 100vh; display: flex; align-items: center; justify-content: center;
  background: linear-gradient(135deg, #1f3a5f 0%, #2f537e 60%, #3d6a9c 100%);
```

```
}
.login-card {
  width: 380px; background: #fff; border-radius: 16px; padding: 40px 36px;
  box-shadow: 0 18px 50px rgba(0, 0, 0, .25); text-align: center;
}
.login-logo { font-size: 52px; }
.login-title { font-size: 22px; margin: 10px 0 4px; color: var(--primary); }
.login-sub { color: var(--text-dim); font-size: 13px; margin-bottom: 22px; }
.login-card .form-label { text-align: left; }
.login-tip { margin-top: 16px; font-size: 12px; color: var(--text-dim); }

/* ---------- 布局 ---------- */
#app { display: flex; min-height: 100vh; }
.sidebar {
  width: 218px; flex-shrink: 0; background: var(--primary); color: #dfe8f3;
  display: flex; flex-direction: column; position: sticky; top: 0; height: 100vh;
}
.brand { display: flex; gap: 10px; align-items: center; padding: 20px 18px 16px; }
.brand-logo { font-size: 30px; }
.brand-name { font-size: 15px; font-weight: 700; color: #fff; }
.brand-ver { font-size: 12px; color: #9db4d0; }
.nav { display: flex; flex-direction: column; gap: 2px; padding: 6px 12px; flex: 1; }
.nav-item {
  display: flex; align-items: center; gap: 10px; padding: 10px 12px;
  border-radius: 8px; color: #c7d6e8; font-size: 14px; transition: all .15s;
}
.nav-item:hover { background: rgba(255, 255, 255, .08); color: #fff; }
.nav-item.active { background: var(--accent); color: #fff; font-weight: 600; }
.nav-icon { font-size: 17px; width: 24px; text-align: center; }
.sidebar-foot { padding: 14px; border-top: 1px solid rgba(255, 255, 255, .12); }
.user-chip { font-size: 13px; margin-bottom: 8px; color: #eaf0f8; }
.main { flex: 1; min-width: 0; display: flex; flex-direction: column; }
.view { padding: 24px 28px; flex: 1; }
.foot { text-align: center; color: var(--text-dim); font-size: 12px; padding: 14px; }

/* ---------- 页面标题区 ---------- */
.page-head { display: flex; align-items: center; justify-content: space-between; margin-bottom:
18px; }
.page-title { font-size: 21px; font-weight: 700; color: var(--primary); }
.page-desc { color: var(--text-dim); font-size: 13px; margin-top: 2px; }
.page-actions { display: flex; gap: 10px; align-items: center; }

/* ---------- 卡片与栅格 ---------- */
.card {
  background: var(--card); border-radius: var(--radius);
  box-shadow: var(--shadow); padding: 18px 20px; margin-bottom: 18px;
  border: 1px solid var(--border);
}
.card-title { font-size: 15px; font-weight: 700; margin-bottom: 12px; color: var(--primary); }
.grid { display: grid; gap: 16px; }
.grid-2 { grid-template-columns: repeat(2, 1fr); }
.grid-3 { grid-template-columns: repeat(3, 1fr); }
.grid-4 { grid-template-columns: repeat(4, 1fr); }
@media (max-width: 1100px) { .grid-4 { grid-template-columns: repeat(2, 1fr); } }

/* ---------- 指标卡 ---------- */
.stat-card { text-align: left; position: relative; overflow: hidden; }
.stat-card::before {
  content: ""; position: absolute; left: 0; top: 0; bottom: 0; width: 4px;
  background: var(--accent);
}
.stat-label { font-size: 13px; color: var(--text-dim); }
```

```
.stat-value { font-size: 30px; font-weight: 800; color: var(--primary); margin: 4px 0; }
.stat-value small { font-size: 14px; font-weight: 400; color: var(--text-dim); }
.stat-hint { font-size: 12px; color: var(--text-dim); }
.stat-card.blue::before { background: #3d7bd6; }
.stat-card.green::before { background: var(--success); }
.stat-card.orange::before { background: var(--accent); }
.stat-card.red::before { background: var(--danger); }

/* ---------- 表格 ---------- */
.table-wrap { overflow-x: auto; }
table.data { width: 100%; border-collapse: collapse; font-size: 14px; }
table.data th {
  text-align: left; padding: 10px 12px; color: var(--text-dim);
  font-weight: 600; font-size: 13px; border-bottom: 2px solid var(--border);
  white-space: nowrap;
}
table.data td { padding: 10px 12px; border-bottom: 1px solid var(--border); vertical-align:
middle; }
table.data tr:hover td { background: #f8fafd; }
.table-ops { white-space: nowrap; }
.link-btn {
  background: none; border: none; color: var(--primary-light);
  cursor: pointer; font-size: 13px; margin-right: 10px; padding: 2px 0;
}
.link-btn:hover { color: var(--accent); text-decoration: underline; }
.link-btn.danger { color: var(--danger); }

/* ---------- 标签徽标 ---------- */
.badge {
  display: inline-block; padding: 2px 10px; border-radius: 20px;
  font-size: 12px; font-weight: 600; white-space: nowrap;
}
.badge.gray { background: #eef1f6; color: #5b6b82; }
.badge.blue { background: #e3eefc; color: #2b63b8; }
.badge.green { background: #dcf3e8; color: #177a4e; }
.badge.orange { background: var(--accent-light); color: #c96a1b; }
.badge.red { background: #fbe3e3; color: #c0392b; }
.badge.purple { background: #ece7f9; color: #6a4fc7; }

/* ---------- 表单 ---------- */
.form-label { display: block; font-size: 13px; color: var(--text-dim); margin: 12px 0 6px; font-
weight: 600; }
.form-input, .form-select, .form-textarea {
  width: 100%; padding: 9px 12px; border: 1px solid var(--border);
  border-radius: 8px; font-size: 14px; font-family: inherit; background: #fff;
  transition: border-color .15s;
}
.form-input:focus, .form-select:focus, .form-textarea:focus {
  outline: none; border-color: var(--primary-light);
  box-shadow: 0 0 0 3px rgba(47, 83, 126, .12);
}
.form-textarea { min-height: 84px; resize: vertical; }
.form-row { display: grid; grid-template-columns: 1fr 1fr; gap: 0 16px; }
.form-error {
  background: #fdecec; color: #c0392b; border: 1px solid #f5c6c6;
  border-radius: 8px; padding: 8px 12px; font-size: 13px; margin-bottom: 12px;
}
.form-hint { font-size: 12px; color: var(--text-dim); margin-top: 4px; }

/* ---------- 按钮 ---------- */
.btn {
  display: inline-block; padding: 9px 20px; border-radius: 8px; border: 1px solid var(--border);
  background: #fff; color: var(--text); font-size: 14px; cursor: pointer;
```

```
  transition: all .15s; font-family: inherit;
}
.btn:hover { border-color: var(--primary-light); color: var(--primary); }
.btn-primary { background: var(--primary); border-color: var(--primary); color: #fff; font-
weight: 600; }
.btn-primary:hover { background: var(--primary-light); color: #fff; }
.btn-accent { background: var(--accent); border-color: var(--accent); color: #fff; font-weight:
600; }
.btn-accent:hover { background: #d97622; color: #fff; }
.btn-danger { background: var(--danger); border-color: var(--danger); color: #fff; }
.btn-ghost { background: transparent; border-color: rgba(255,255,255,.35); color: #dfe8f3; }
.btn-ghost:hover { border-color: #fff; color: #fff; }
.btn-sm { padding: 5px 12px; font-size: 13px; }
.btn-block { width: 100%; margin-top: 18px; }

/* ---------- 筛选条 ---------- */
.filter-bar {
  display: flex; gap: 10px; align-items: center; margin-bottom: 14px; flex-wrap: wrap;
}
.filter-bar .form-input, .filter-bar .form-select { width: auto; }
.search-input { width: 240px !important; }

/* ---------- 阶段看板 ---------- */
.stage-board { display: grid; grid-template-columns: repeat(6, 1fr); gap: 12px; }
.stage-col { background: #f8fafd; border: 1px solid var(--border); border-radius: 8px; padding:
10px; min-height: 120px; }
.stage-head { font-size: 13px; font-weight: 700; margin-bottom: 8px; display: flex; justify-
content: space-between; align-items: center; }
.stage-count { background: var(--primary); color: #fff; border-radius: 12px; padding: 0 8px;
font-size: 12px; }
.job-mini {
  background: #fff; border: 1px solid var(--border); border-radius: 6px;
  padding: 8px 10px; margin-bottom: 8px; font-size: 13px; cursor: pointer;
}
.job-mini:hover { border-color: var(--accent); }
.job-mini .c { font-weight: 700; }
.job-mini .p { color: var(--text-dim); font-size: 12px; }

/* ---------- 漏斗图 ---------- */
.funnel { display: flex; flex-direction: column; gap: 8px; }
.funnel-row { display: flex; align-items: center; gap: 12px; }
.funnel-label { width: 70px; text-align: right; font-size: 13px; color: var(--text-dim); }
.funnel-bar {
  height: 30px; border-radius: 6px; background: linear-gradient(90deg, var(--primary), var(--
primary-light));
  display: flex; align-items: center; justify-content: flex-end; padding-right: 10px;
  color: #fff; font-size: 13px; font-weight: 700; min-width: 46px; transition: width .4s;
}
.funnel-rate { font-size: 12px; color: var(--text-dim); width: 90px; }

/* ---------- 条形图（纯 CSS） ---------- */
.bar-chart { display: flex; flex-direction: column; gap: 10px; }
.bar-row { display: grid; grid-template-columns: 90px 1fr 110px; align-items: center; gap: 10px;
font-size: 13px; }
.bar-track { background: #eef1f6; border-radius: 6px; height: 18px; overflow: hidden; }
.bar-fill { height: 100%; border-radius: 6px; background: linear-gradient(90deg, #3d7bd6,
#6aa3f0); transition: width .4s; }
.bar-fill.green { background: linear-gradient(90deg, #22a06b, #4cc38d); }
.bar-fill.orange { background: linear-gradient(90deg, #f0852c, #f5ac63); }
.bar-val { color: var(--text-dim); font-size: 12px; }

/* ---------- SVG 折线图 ---------- */
.line-chart { width: 100%; height: 220px; }
.line-chart .axis { stroke: #d5dce8; stroke-width: 1; }
.line-chart .grid-line { stroke: #eef1f6; stroke-width: 1; }
.line-chart .tick-label { font-size: 10px; fill: #8a99ad; }
.line-chart .data-line { fill: none; stroke: var(--accent); stroke-width: 2.5; }
.line-chart .data-area { fill: rgba(240, 133, 44, .12); }
.line-chart .data-dot { fill: var(--accent); }
```

```
/* ---------- 雷达图 ---------- */
.radar-wrap { display: flex; gap: 24px; align-items: center; }
.radar-svg { width: 300px; height: 300px; flex-shrink: 0; }
.radar-legend { font-size: 13px; }
.radar-legend li { margin-bottom: 8px; list-style: none; }
.radar-dot { display: inline-block; width: 10px; height: 10px; border-radius: 50%; margin-right:
8px; }

/* ---------- 待办 ---------- */
.todo-item {
  display: flex; align-items: center; gap: 12px; padding: 12px 14px;
  background: #fff; border: 1px solid var(--border); border-radius: 8px; margin-bottom: 10px;
}
.todo-item.done { opacity: .55; }
.todo-item.done .todo-title { text-decoration: line-through; }
.todo-check {
  width: 22px; height: 22px; border-radius: 50%; border: 2px solid #b9c6d8;
  cursor: pointer; flex-shrink: 0; display: flex; align-items: center; justify-content: center;
  color: #fff; font-size: 13px;
}
.todo-check.on { background: var(--success); border-color: var(--success); }
.todo-main { flex: 1; min-width: 0; }
.todo-title { font-size: 14px; font-weight: 600; }
.todo-meta { font-size: 12px; color: var(--text-dim); margin-top: 2px; display: flex; gap: 10px;
flex-wrap: wrap; }
.todo-ops { display: flex; gap: 4px; }
.top3-flag { color: var(--accent); font-weight: 700; font-size: 12px; }
.overdue-tag { color: var(--danger); font-weight: 700; font-size: 12px; }

/* ---------- 考试倒计时 ---------- */
.exam-card { display: flex; align-items: center; gap: 16px; }
.exam-date-box {
  width: 74px; height: 74px; border-radius: 10px; background: var(--primary);
  color: #fff; display: flex; flex-direction: column; align-items: center; justify-content:
center;
  flex-shrink: 0;
}
.exam-date-box .d { font-size: 26px; font-weight: 800; line-height: 1; }
.exam-date-box .m { font-size: 12px; opacity: .85; }
.exam-date-box.urgent { background: var(--danger); }
.exam-info { flex: 1; }
.exam-name { font-size: 15px; font-weight: 700; }
.exam-meta { font-size: 13px; color: var(--text-dim); margin-top: 2px; }
.countdown { font-size: 13px; font-weight: 700; color: var(--accent); white-space: nowrap; }
.countdown.passed { color: var(--text-dim); font-weight: 400; }

/* ---------- 资料卡片 ---------- */
.material-grid { display: grid; grid-template-columns: repeat(3, 1fr); gap: 14px; }
.material-card { border: 1px solid var(--border); border-radius: 8px; padding: 14px; background:
#fff; }
.material-card:hover { box-shadow: var(--shadow); }
.material-title { font-size: 14px; font-weight: 700; margin-bottom: 6px; }
.material-meta { font-size: 12px; color: var(--text-dim); display: flex; gap: 8px; flex-wrap:
wrap; margin-bottom: 8px; }
.material-notes { font-size: 13px; color: var(--text-dim); margin-bottom: 10px; }
.stars { color: var(--accent); letter-spacing: 2px; font-size: 14px; }

/* ---------- 弹窗 ---------- */
.modal-mask {
  position: fixed; inset: 0; background: rgba(20, 32, 48, .5);
  display: flex; align-items: center; justify-content: center; z-index: 100;
}
.modal { width: 560px; max-width: 92vw; max-height: 88vh; background: #fff; border-radius: 12px;
display: flex; flex-direction: column; }
.modal-head { display: flex; justify-content: space-between; align-items: center; padding: 16px
20px; border-bottom: 1px solid var(--border); }
.modal-title { font-size: 16px; font-weight: 700; color: var(--primary); }
```

```
.modal-x { background: none; border: none; font-size: 16px; cursor: pointer; color: var(--text-dim); }
.modal-body { padding: 6px 20px 16px; overflow-y: auto; }
.modal-foot { padding: 14px 20px; border-top: 1px solid var(--border); display: flex; justify-content: flex-end; gap: 10px; }

/* ---------- 轻提示 ---------- */
.toast {
  position: fixed; bottom: 30px; left: 50%; transform: translateX(-50%);
  background: rgba(31, 58, 95, .95); color: #fff; padding: 10px 22px;
  border-radius: 24px; font-size: 14px; z-index: 200; box-shadow: 0 6px 20px rgba(0,0,0,.2);
}

/* ---------- 空状态 ---------- */
.empty { text-align: center; padding: 40px 20px; color: var(--text-dim); }
.empty .big { font-size: 44px; margin-bottom: 8px; }

/* ---------- 设置页 ---------- */
.setting-row { display: flex; align-items: center; justify-content: space-between; padding: 14px 0; border-bottom: 1px solid var(--border); }
.setting-row:last-child { border-bottom: none; }
.setting-label { font-size: 14px; font-weight: 600; }
.setting-desc { font-size: 12px; color: var(--text-dim); margin-top: 2px; }
.about-box { background: #f8fafd; border-radius: 8px; padding: 16px; font-size: 13px; color: var(--text-dim); line-height: 2; }

/* ---------- 打印 ---------- */
@media print {
  .sidebar, .page-actions, .modal-mask, .toast, .foot { display: none !important; }
  .main { margin: 0; }
}
```

——— static/js/app.js ———

```
/* 求职备考管理软件 V1.0 · 前端核心
   职责：API 封装、哈希路由、登录态管理、弹窗/轻提示、通用工具函数。
   各业务视图实现在 view_*.js 中，通过 window.Views 注册。 */
'use strict';

/* ================= API 客户端 ================= */
const API = {
  async request(method, path, body) {
    const opts = { method, headers: { 'Content-Type': 'application/json' } };
    if (body !== undefined) opts.body = JSON.stringify(body);
    const resp = await fetch(path, opts);
    let data = null;
    try { data = await resp.json(); } catch (e) { /* 非 JSON 响应 */ }
    if (resp.status === 401 && data && data.code === 'NOT_LOGGED_IN') {
      // 会话失效：回到登录页
      AppState.user = null;
      Router.go('login');
      throw new Error('登录已过期，请重新登录');
    }
    if (!resp.ok) {
      throw new Error((data && data.error) || ('请求失败(' + resp.status + ')'));
    }
    return data;
  },
  get(path)    { return this.request('GET', path); },
  post(path, b){ return this.request('POST', path, b); },
  put(path, b) { return this.request('PUT', path, b); },
  del(path)    { return this.request('DELETE', path); },
};

/* ================= 全局状态 ================= */
const AppState = {
  user: null,
```

```
  // 业务字典（与后端 models.py 保持一致）
  JOB_STAGES: ['待投递', '已投递', '笔试', '面试', 'Offer', '已拒绝'],
  PRACTICE_MODULES: ['图形推理', '逻辑判断', '数量分析', '资料分析', '言语理解', '常识判断'],
  MATERIAL_CATEGORIES: ['行测', '申论', '专业课', '面试', '简历', '其他'],
  MATERIAL_STATUS: ['未开始', '学习中', '已完成'],
  TODO_CATEGORIES: ['求职', '备考', '生活', '其他'],
  PRIORITIES: ['高', '中', '低'],
};

/* ================= 工具函数 ================= */
const Util = {
  // HTML 转义，防止 XSS
  esc(s) {
    return String(s == null ? '' : s)
      .replace(/&/g, '&amp;').replace(/</g, '&lt;')
      .replace(/>/g, '&gt;').replace(/"/g, '&quot;')
      .replace(/'/g, '&#39;');
  },
  // 日期格式化：YYYY-MM-DD → M月D日
  md(dateStr) {
    if (!dateStr) return '—';
    const m = /^(\d{4})-(\d{2})-(\d{2})/.exec(dateStr);
    if (!m) return dateStr;
    return parseInt(m[2], 10) + '月' + parseInt(m[3], 10) + '日';
  },
  today() {
    const d = new Date();
    const p = n => String(n).padStart(2, '0');
    return d.getFullYear() + '-' + p(d.getMonth() + 1) + '-' + p(d.getDate());
  },
  // 倒计时描述
  countdown(dateStr) {
    if (!dateStr) return null;
    const t = new Date(dateStr + 'T00:00:00');
    if (isNaN(t)) return null;
    const now = new Date(); now.setHours(0, 0, 0, 0);
    const diff = Math.round((t - now) / 86400000);
    if (diff < 0) return { text: '已过去 ' + (-diff) + ' 天', passed: true };
    if (diff === 0) return { text: '就是今天', urgent: true };
    return { text: '还剩 ' + diff + ' 天', urgent: diff <= 7 };
  },
  // 阶段徽标颜色映射
  stageBadge(stage) {
    const map = {
      '待投递': 'gray', '已投递': 'blue', '笔试': 'purple',
      '面试': 'orange', 'Offer': 'green', '已拒绝': 'red',
    };
    return '<span class="badge ' + (map[stage] || 'gray') + '">' +
           this.esc(stage) + '</span>';
  },
  priorityBadge(p) {
    const map = { '高': 'red', '中': 'orange', '低': 'gray' };
    return '<span class="badge ' + (map[p] || 'gray') + '">' +
           this.esc(p) + '优先级</span>';
  },
  statusBadge(s) {
    const map = { '未开始': 'gray', '学习中': 'blue', '已完成': 'green' };
    return '<span class="badge ' + (map[s] || 'gray') + '">' +
           this.esc(s) + '</span>';
  },
  // 百分比显示
```

```
  pct(v) { return (Math.round(v * 1000) / 10) + '%'; },
  // 星级显示
  stars(n) {
    n = Math.max(0, Math.min(5, parseInt(n, 10) || 0));
    return '<span class="stars">' + '★'.repeat(n) + '☆'.repeat(5 - n) + '</span>';
  },
  // 下拉选项生成
  options(list, selected) {
    return list.map(v =>
      '<option value="' + this.esc(v) + '"' +
      (v === selected ? ' selected' : '') + '>' + this.esc(v) + '</option>'
    ).join('');
  },
  // 防抖
  debounce(fn, ms) {
    let t = null;
    return function (...args) {
      clearTimeout(t);
      t = setTimeout(() => fn.apply(this, args), ms || 300);
    };
  },
};

/* ================= 轻提示 ================= */
const Toast = {
  timer: null,
  show(msg, ms) {
    const el = document.getElementById('toast');
    el.textContent = msg;
    el.classList.remove('hidden');
    clearTimeout(this.timer);
    this.timer = setTimeout(() => el.classList.add('hidden'), ms || 2200);
  },
};

/* ================= 通用弹窗 ================= */
const Modal = {
  okHandler: null,
  open(title, bodyHTML, onOk) {
    document.getElementById('modal-title').textContent = title;
    document.getElementById('modal-body').innerHTML = bodyHTML;
    document.getElementById('modal-mask').classList.remove('hidden');
    this.okHandler = onOk || null;
    const first = document.querySelector('#modal-body .form-input, #modal-body .form-select');
    if (first) first.focus();
  },
  close() {
    document.getElementById('modal-mask').classList.add('hidden');
    this.okHandler = null;
  },
  // 读取弹窗内表单：按 name 属性收集值
  collect() {
    const data = {};
    document.querySelectorAll('#modal-body [name]').forEach(el => {
      if (el.type === 'checkbox') data[el.name] = el.checked ? 1 : 0;
      else data[el.name] = el.value;
    });
    return data;
  },
  // 快捷表单弹窗：fields 为 [{name,label,type,options,value,required,placeholder}]
  form(title, fields, onOk, okText) {
```

```
    const body = fields.map(f => {
      const req = f.required ? ' <span style="color:#e05252">*</span>' : '';
      let input = '';
      if (f.type === 'select') {
        input = '<select name="' + f.name + '" class="form-select">' +
                Util.options(f.options, f.value) + '</select>';
      } else if (f.type === 'textarea') {
        input = '<textarea name="' + f.name + '" class="form-textarea" placeholder="' +
                Util.esc(f.placeholder || '') + '">' + Util.esc(f.value || '') + '</textarea>';
      } else if (f.type === 'checkbox') {
        input = '<label style="display:flex;align-items:center;gap:8px;font-size:14px">' +
                '<input type="checkbox" name="' + f.name + '" ' +
                (f.value ? ' checked' : '') + '> ' + Util.esc(f.checkLabel || '') + '</label>';
        return '<div style="margin:12px 0">' + input + '</div>';
      } else {
        input = '<input name="' + f.name + '" type="' + (f.type || 'text') +
                '" class="form-input" value="' + Util.esc(f.value || '') +
                '" placeholder="' + Util.esc(f.placeholder || '') + '">';
      }
      return '<label class="form-label">' + Util.esc(f.label) + req + '</label>' + input;
    }).join('');
    this.open(title, body, onOk);
    if (okText) document.getElementById('modal-ok').textContent = okText;
  },
  confirm(title, text, onOk) {
    this.open(title, '<p style="font-size:14px;padding:8px 0">' +
              Util.esc(text) + '</p>', onOk);
    document.getElementById('modal-ok').textContent = '确定删除';
  },
};

/* ================ 路由 ================ */
const Router = {
  routes: {},
  // 注册视图：Views 在各自文件中调用 Router.register('jobs', {...})
  register(name, view) { this.routes[name] = view; },
  current: null,
  go(name) {
    if (name === 'login') { location.hash = '#/login'; return; }
    location.hash = '#/' + name;
  },
  parse() {
    const h = location.hash || '#/dashboard';
    const m = /^#\/(\w+)/.exec(h);
    return (m && m[1]) || 'dashboard';
  },
  async render() {
    const name = this.parse();
    const loginPage = document.getElementById('login-page');
    const app = document.getElementById('app');
    // 未登录：强制登录页
    if (!AppState.user) {
      if (name !== 'login') { location.hash = '#/login'; return; }
      loginPage.classList.remove('hidden');
      app.classList.add('hidden');
      return;
    }
    if (name === 'login') { location.hash = '#/dashboard'; return; }
    loginPage.classList.add('hidden');
    app.classList.remove('hidden');
    // 高亮导航
```

```
    document.querySelectorAll('#main-nav .nav-item').forEach(a => {
      a.classList.toggle('active', a.dataset.route === name);
    });
    const view = this.routes[name] || this.routes.dashboard;
    this.current = name;
    const container = document.getElementById('view');
    container.innerHTML = '<div class="empty"><div class="big">⏳</div>加载中…</div>';
    try {
      container.innerHTML = await view.render();
      if (view.afterRender) view.afterRender(container);
    } catch (e) {
      container.innerHTML = '<div class="empty"><div class="big">⚠️</div>' +
        Util.esc(e.message) + '</div>';
    }
    window.scrollTo(0, 0);
  },
};

/* ================ 登录逻辑 ================ */
function bindLogin() {
  const doLogin = async () => {
    const username = document.getElementById('login-username').value.trim();
    const password = document.getElementById('login-password').value;
    const errBox = document.getElementById('login-error');
    errBox.classList.add('hidden');
    if (!username || !password) {
      errBox.textContent = '请输入用户名和密码';
      errBox.classList.remove('hidden');
      return;
    }
    try {
      const data = await API.post('/api/login', { username, password });
      AppState.user = data.user;
      document.getElementById('username-label').textContent = data.user.username;
      Toast.show('登录成功，欢迎回来');
      Router.go('dashboard');
    } catch (e) {
      errBox.textContent = e.message;
      errBox.classList.remove('hidden');
    }
  };
  document.getElementById('login-btn').addEventListener('click', doLogin);
  document.getElementById('login-password').addEventListener('keydown', e => {
    if (e.key === 'Enter') doLogin();
  });
  document.getElementById('logout-btn').addEventListener('click', async () => {
    await API.post('/api/logout');
    AppState.user = null;
    Router.go('login');
  });
}

/* ================ 弹窗事件绑定 ================ */
function bindModal() {
  document.getElementById('modal-close').addEventListener('click', () => Modal.close());
  document.getElementById('modal-cancel').addEventListener('click', () => Modal.close());
  document.getElementById('modal-mask').addEventListener('click', e => {
    if (e.target.id === 'modal-mask') Modal.close();
  });
  document.getElementById('modal-ok').addEventListener('click', async () => {
    const fn = Modal.okHandler;
```

```
    if (!fn) { Modal.close(); return; }
    try {
      const data = Modal.collect();
      const keep = await fn(data);
      if (keep !== false) {
        Modal.close();
        document.getElementById('modal-ok').textContent = '确定';
      }
    } catch (e) {
      Toast.show(e.message);
    }
  });
  document.addEventListener('keydown', e => {
    if (e.key === 'Escape' &&
        !document.getElementById('modal-mask').classList.contains('hidden')) {
      Modal.close();
    }
  });
}

/* ================= 启动 ================= */
window.Views = {};
document.addEventListener('DOMContentLoaded', async () => {
  bindLogin();
  bindModal();
  // 恢复会话
  try {
    const data = await API.get('/api/me');
    if (data.user) {
      AppState.user = data.user;
      document.getElementById('username-label').textContent = data.user.username;
    }
  } catch (e) { /* 后端未启动时保持登录页 */ }
  window.addEventListener('hashchange', () => Router.render());
  Router.render();
});
```

——— static/js/view_dashboard.js ———

```
/* 求职备考管理软件 V1.0 · 数据总览视图
   展示：核心指标卡、求职漏斗、笔试趋势折线图、模块正确率条形图、
         今日三件事、临近考试、近期截止投递。 */
'use strict';

Router.register('dashboard', {
  async render() {
    const data = await API.get('/api/stats/overview');
    const s = data.stats;
    return `
    <div class="page-head">
      <div>
        <div class="page-title">数据总览</div>
        <div class="page-desc">求职与备考的关键指标一览 · 数据更新于 ${Util.esc(s.generated_at)}</div>
      </div>
    </div>

    <div class="grid grid-4">
      ${statCard('blue', '投递总数', s.jobs.total + '<small> 家</small>',
                 '进行中 ' + s.jobs.active + ' 家 · Offer 率 ' + Util.pct(s.jobs.offer_rate))}
      ${statCard('green', '累计刷题', s.practice.total_questions + '<small> 题</small>',
                 '总体正确率 ' + Util.pct(s.practice.overall_accuracy) +
                 ' · 近7天 ' + s.practice.week_questions + ' 题')}
      ${statCard('orange', '待办事项', s.todos.undone + '<small> 项未完成</small>',
```

```
                s.todos.overdue > 0
                  ? '其中 <b style="color:#e05252">' + s.todos.overdue + ' 项已逾期</b>'
                  : '暂无逾期事项，继续保持')}
      ${statCard('red', '备考资料', s.materials.total + '<small> 份</small>',
                '已完成 ' + Util.pct(s.materials.done_rate))}
    </div>

    <div class="grid grid-2">
      <div class="card">
        <div class="card-title">🎯 求职漏斗</div>
        ${funnelHTML(s.jobs.by_stage)}
        <div style="margin-top:12px;font-size:13px;color:#6b7a90">
          ${s.practice.weakest_module
            ? '💡 笔试薄弱模块：<b>' + Util.esc(s.practice.weakest_module) +
              '</b>，建议优先补强'
            : '💡 暂无笔试记录，去记录一次练习吧'}
        </div>
      </div>
      <div class="card">
        <div class="card-title">📈 近 7 天刷题趋势</div>
        ${lineChart(s.practice.daily_trend)}
      </div>
    </div>

    <div class="grid grid-2">
      <div class="card">
        <div class="card-title">📊 各模块正确率</div>
        ${moduleBars(s.practice.by_module)}
      </div>
      <div class="card">
        <div class="card-title">⭐ 今日三件事</div>
        ${top3HTML(s.todos.top3)}
      </div>
    </div>

    <div class="grid grid-2">
      <div class="card">
        <div class="card-title">📅 临近考试（30 天内）</div>
        ${upcomingExamsHTML(s.exams.upcoming)}
      </div>
      <div class="card">
        <div class="card-title">⏰ 投递截止提醒（14 天内）</div>
        ${deadlinesHTML(s.jobs.upcoming_deadlines)}
      </div>
    </div>`;
  },
});

/* ---------- 局部渲染函数 ---------- */

function statCard(color, label, value, hint) {
  return `<div class="card stat-card ${color}">
    <div class="stat-label">${label}</div>
    <div class="stat-value">${value}</div>
    <div class="stat-hint">${hint}</div>
  </div>`;
}

function funnelHTML(byStage) {
  const order = ['待投递', '已投递', '笔试', '面试', 'Offer'];
  const max = Math.max(1, ...order.map(s => byStage[s] || 0));
```

```
  let prev = null;
  const rows = order.map(stage => {
    const c = byStage[stage] || 0;
    const width = Math.max(8, Math.round((c / max) * 100));
    const rate = prev === null ? '—'
      : (prev > 0 ? Util.pct(c / prev) : '0%');
    prev = c;
    return `<div class="funnel-row">
      <div class="funnel-label">${stage}</div>
      <div class="funnel-bar" style="width:${width}%">${c}</div>
      <div class="funnel-rate">转化 ${rate}</div>
    </div>`;
  }).join('');
  return `<div class="funnel">${rows}</div>`;
}

// SVG 折线图：每日刷题量
function lineChart(trend) {
  if (!trend || !trend.length) {
    return '<div class="empty">暂无数据</div>';
  }
  const W = 560, H = 220, P = { l: 36, r: 12, t: 14, b: 30 };
  const maxV = Math.max(1, ...trend.map(d => d.total));
  const n = trend.length;
  const x = i => P.l + (i * (W - P.l - P.r)) / Math.max(1, n - 1);
  const y = v => H - P.b - (v / maxV) * (H - P.t - P.b);
  const pts = trend.map((d, i) => x(i).toFixed(1) + ',' + y(d.total).toFixed(1)).join(' ');
  const area = `${P.l},${H - P.b} ${pts} ${x(n - 1).toFixed(1)},${H - P.b}`;
  // 网格线
  let grid = '';
  for (let g = 0; g <= 4; g++) {
    const gy = P.t + (g * (H - P.t - P.b)) / 4;
    const val = Math.round(maxV * (1 - g / 4));
    grid += `<line class="grid-line" x1="${P.l}" y1="${gy}" x2="${W - P.r}" y2="${gy}"/>` +
            `<text class="tick-label" x="${P.l - 6}" y="${gy + 4}"
text-anchor="end">${val}</text>`;
  }
  const dots = trend.map((d, i) =>
    `<circle class="data-dot" cx="${x(i).toFixed(1)}" cy="${y(d.total).toFixed(1)}" r="4">` +
    `<title>${Util.esc(d.date)}：${d.total}题，正确率${Util.pct(d.accuracy)}</title></circle>`
  ).join('');
  const labels = trend.map((d, i) => {
    if (n > 10 && i % 2 === 1) return '';
    return `<text class="tick-label" x="${x(i).toFixed(1)}" y="${H - 10}" text-anchor="middle">`
+
           `${Util.esc(d.date.slice(5))}</text>`;
  }).join('');
  return `<svg class="line-chart" viewBox="0 0 ${W} ${H}">
    ${grid}
    <polygon class="data-area" points="${area}"/>
    <polyline class="data-line" points="${pts}"/>
    ${dots}${labels}
  </svg>`;
}

function moduleBars(byModule) {
  const mods = AppState.PRACTICE_MODULES;
  const rows = mods.map(m => {
    const v = byModule[m];
    const acc = v ? v.accuracy : 0;
    const width = Math.round(acc * 100);
    const cls = acc >= 0.8 ? 'green' : (acc >= 0.6 ? '' : 'orange');
    const val = v ? `${v.correct}/${v.total}题 · ${Util.pct(acc)}` : '暂无记录';
```

```
    return `<div class="bar-row">
      <div>${Util.esc(m)}</div>
      <div class="bar-track"><div class="bar-fill ${cls}" style="width:${width}%"></div></div>
      <div class="bar-val">${val}</div>
    </div>`;
  }).join('');
  return `<div class="bar-chart">${rows}</div>`;
}

function top3HTML(items) {
  if (!items.length) {
    return '<div class="empty"><div class="big">🎯</div>今日三件事还没有安排<br>' +
           '<span style="font-size:13px">去「待办清单」把最重要的 3 件事标出来吧</span></div>';
  }
  return items.map((t, i) => `
    <div class="todo-item">
      <div style="font-size:20px">${['🥇', '🥈', '🥉'][i] || '▫'}</div>
      <div class="todo-main">
        <div class="todo-title">${Util.esc(t.title)}</div>
        <div class="todo-meta"><span>${Util.esc(t.category)}</span>
        ${t.due_date ? '<span>截止 ' + Util.md(t.due_date) + '</span>' : ''}</div>
      </div>
      ${Util.priorityBadge(t.priority)}
    </div>`).join('') +
    '<div style="text-align:right;margin-top:8px"><a href="#/todos">去管理待办 →</a></div>';
}

function upcomingExamsHTML(items) {
  if (!items.length) {
    return '<div class="empty"><div class="big">📆</div>30 天内暂无考试安排</div>';
  }
  return items.map(e => {
    const d = /^(\d{4})-(\d{2})-(\d{2})/.exec(e.exam_date) || [];
    return `<div class="exam-card" style="margin-bottom:12px">
      <div class="exam-date-box ${e.days_left <= 7 ? 'urgent' : ''}">
        <div class="d">${d[3] ? parseInt(d[3], 10) : '—'}</div>
        <div class="m">${d[2] ? parseInt(d[2], 10) + '月' : ''}</div>
      </div>
      <div class="exam-info">
        <div class="exam-name">${Util.esc(e.name)}</div>
        <div class="exam-meta">${Util.esc(e.location || '地点待定')}</div>
      </div>
      <div class="countdown ${e.days_left <= 7 ? '' : ''}">还剩 ${e.days_left} 天</div>
    </div>`;
  }).join('');
}

function deadlinesHTML(items) {
  if (!items.length) {
    return '<div class="empty"><div class="big">🎉</div>14 天内没有截止的投递</div>';
  }
  return `<table class="data"><thead><tr>
      <th>公司</th><th>岗位</th><th>截止日期</th><th>状态</th></tr></thead><tbody>` +
    items.slice(0, 6).map(j => {
      const cd = Util.countdown(j.deadline);
      return `<tr><td>${Util.esc(j.company)}</td><td>${Util.esc(j.position)}</td>` +
        `<td>${Util.md(j.deadline)}</td>` +
        `<td>${cd ? '<span class="' + (cd.urgent ? 'overdue-tag' : '') + '">' +
          Util.esc(cd.text) + '</span>' : '—'}</td></tr>`;
    }).join('') + '</tbody></table>';
}
```

——— static/js/view_jobs.js ———

```
/* 求职备考管理软件 V1.0 · 求职投递视图
   功能：投递列表（筛选/搜索）、阶段看板、新增/编辑/删除、阶段流转。 */
'use strict';

Router.register('jobs', {
  _stage: '',
  _keyword: '',

  async render() {
    const data = await API.get('/api/jobs');
    this._all = data.items;
    return `
    <div class="page-head">
      <div>
        <div class="page-title">求职投递</div>
        <div class="page-desc">跟踪每一次投递的全流程：从网申到 Offer</div>
      </div>
      <div class="page-actions">
        <button class="btn btn-primary" id="job-add">＋ 新增投递</button>
      </div>
    </div>

    <div class="card">
      <div class="card-title">📌 阶段看板</div>
      <div class="stage-board" id="stage-board"></div>
    </div>

    <div class="card">
      <div class="filter-bar">
        <select id="job-stage-filter" class="form-select">
          <option value="">全部阶段</option>
          ${Util.options(AppState.JOB_STAGES, '')}
        </select>
        <input id="job-keyword" class="form-input search-input" placeholder="搜索公司 / 岗位 / 城
市">
        <span style="font-size:13px;color:#6b7a90" id="job-count"></span>
      </div>
      <div class="table-wrap"><table class="data">
        <thead><tr>
          <th>公司</th><th>岗位</th><th>城市</th><th>阶段</th>
          <th>投递日期</th><th>截止日期</th><th>优先级</th><th>操作</th>
        </tr></thead>
        <tbody id="job-tbody"></tbody>
      </table></div>
      <div id="job-empty"></div>
    </div>`;
  },

  afterRender(root) {
    this.renderBoard();
    this.renderTable();
    root.querySelector('#job-add').addEventListener('click', () => this.openForm());
    root.querySelector('#job-stage-filter').addEventListener('change', e => {
      this._stage = e.target.value;
      this.renderTable();
    });
    const kwInput = root.querySelector('#job-keyword');
    kwInput.addEventListener('input', Util.debounce(() => {
      this._keyword = kwInput.value.trim();
      this.renderTable();
    }));
```

```
  },

  /* ----- 数据筛选 ----- */
  filtered() {
    return this._all.filter(j => {
      if (this._stage && j.stage !== this._stage) return false;
      if (this._keyword) {
        const kw = this._keyword.toLowerCase();
        const hay = (j.company + ' ' + j.position + ' ' + j.city).toLowerCase();
        if (!hay.includes(kw)) return false;
      }
      return true;
    });
  },

  /* ----- 阶段看板 ----- */
  renderBoard() {
    const board = document.getElementById('stage-board');
    const groups = {};
    AppState.JOB_STAGES.forEach(s => { groups[s] = []; });
    this._all.forEach(j => { (groups[j.stage] || groups['待投递']).push(j); });
    board.innerHTML = AppState.JOB_STAGES.map(s => `
      <div class="stage-col">
        <div class="stage-head"><span>${s}</span>
          <span class="stage-count">${groups[s].length}</span></div>
        ${groups[s].slice(0, 4).map(j => `
          <div class="job-mini" data-id="${j.id}">
            <div class="c">${Util.esc(j.company)}</div>
            <div class="p">${Util.esc(j.position)}</div>
          </div>`).join('')}
        ${groups[s].length > 4
          ? '<div style="font-size:12px;color:#6b7a90">还有 ' +
            (groups[s].length - 4) + ' 家…</div>' : ''}
      </div>`).join('');
    board.querySelectorAll('.job-mini').forEach(el => {
      el.addEventListener('click', () => this.openForm(parseInt(el.dataset.id, 10)));
    });
  },

  /* ----- 列表 ----- */
  renderTable() {
    const rows = this.filtered();
    const tb = document.getElementById('job-tbody');
    document.getElementById('job-count').textContent = '共 ' + rows.length + ' 条';
    document.getElementById('job-empty').innerHTML =
      rows.length ? '' : '<div class="empty"><div class="big">💼</div>还没有投递记录，点击右上角新增
一条吧</div>';
    tb.innerHTML = rows.map(j => {
      const cd = j.deadline ? Util.countdown(j.deadline) : null;
      return `<tr>
        <td><b>${Util.esc(j.company)}</b></td>
        <td>${Util.esc(j.position)}</td>
        <td>${Util.esc(j.city || '—')}</td>
        <td>${Util.stageBadge(j.stage)}</td>
        <td>${j.applied_date ? Util.md(j.applied_date) : '—'}</td>
        <td>${j.deadline ? Util.md(j.deadline) +
          (cd && !cd.passed ? ' <span class="' + (cd.urgent ? 'overdue-tag' : '') +
            '" style="font-size:12px">(' + Util.esc(cd.text) + ')</span>' : '') : '—'}</td>
        <td>${Util.priorityBadge(j.priority)}</td>
        <td class="table-ops">
          <button class="link-btn" data-act="advance" data-id="${j.id}">推进阶段</button>
          <button class="link-btn" data-act="edit" data-id="${j.id}">编辑</button>
```

```
        <button class="link-btn danger" data-act="del" data-id="${j.id}">删除</button>
      </td>
    </tr>`;
  }).join('');
  tb.querySelectorAll('button').forEach(btn => {
    const id = parseInt(btn.dataset.id, 10);
    const act = btn.dataset.act;
    btn.addEventListener('click', () => {
      if (act === 'edit') this.openForm(id);
      else if (act === 'del') this.confirmDelete(id);
      else if (act === 'advance') this.advanceStage(id);
    });
  });
},

/* ----- 新增 / 编辑表单 ----- */
openForm(id) {
  const item = id ? this._all.find(j => j.id === id) : null;
  Modal.form(item ? '编辑投递' : '新增投递', [
    { name: 'company', label: '公司名称', required: true,
      value: item ? item.company : '', placeholder: '如：星云科技' },
    { name: 'position', label: '投递岗位', required: true,
      value: item ? item.position : '', placeholder: '如：产品经理' },
    { name: 'city', label: '工作城市', value: item ? item.city : '',
      placeholder: '如：杭州' },
    { name: 'stage', label: '当前阶段', type: 'select',
      options: AppState.JOB_STAGES, value: item ? item.stage : '待投递' },
    { name: 'source', label: '投递渠道', value: item ? item.source : '',
      placeholder: '如：校园招聘 / 招聘网站 / 内推' },
    { name: 'applied_date', label: '投递日期', type: 'date',
      value: item ? (item.applied_date || '') : Util.today() },
    { name: 'deadline', label: '截止日期', type: 'date',
      value: item ? (item.deadline || '') : '' },
    { name: 'salary_range', label: '薪资范围', value: item ? item.salary_range : '',
      placeholder: '如：18-25K' },
    { name: 'priority', label: '优先级', type: 'select',
      options: AppState.PRIORITIES, value: item ? item.priority : '中' },
    { name: 'notes', label: '备注', type: 'textarea',
      value: item ? item.notes : '', placeholder: '面试准备要点、联系人等' },
  ], async data => {
    try {
      if (item) await API.put('/api/jobs/' + id, data);
      else await API.post('/api/jobs', data);
      Toast.show(item ? '已保存修改' : '投递记录已新增');
      await this.refresh();
    } catch (e) { Toast.show(e.message); return false; }
  });
},

/* ----- 阶段推进：按流程顺序进入下一阶段 ----- */
async advanceStage(id) {
  const item = this._all.find(j => j.id === id);
  if (!item) return;
  const order = AppState.JOB_STAGES;
  const idx = order.indexOf(item.stage);
  if (idx < 0 || idx >= order.length - 1) {
    Toast.show('已是最终阶段');
    return;
  }
  const next = order[idx + 1];
  Modal.open('推进阶段',
```

```
      `<p style="font-size:14px">将「${Util.esc(item.company)} · ` +
      `${Util.esc(item.position)}」从 ${Util.stageBadge(item.stage)} ` +
      `推进到 ${Util.stageBadge(next)}? </p>`,
      async () => {
        try {
          await API.put('/api/jobs/' + id, { stage: next });
          Toast.show('已推进到: ' + next);
          await this.refresh();
        } catch (e) { Toast.show(e.message); return false; }
      });
  },

  confirmDelete(id) {
    const item = this._all.find(j => j.id === id);
    Modal.confirm('删除确认',
      '确定删除「' + item.company + ' · ' + item.position + '」的投递记录吗? 此操作不可恢复。',
      async () => {
        try {
          await API.del('/api/jobs/' + id);
          Toast.show('已删除');
          await this.refresh();
        } catch (e) { Toast.show(e.message); return false; }
      });
  },

  async refresh() {
    const data = await API.get('/api/jobs');
    this._all = data.items;
    this.renderBoard();
    this.renderTable();
  },
});
```

——— static/js/view_practice.js ———

```
/* 求职备考管理软件 V1.0 · 笔试练习视图
   功能: 练习记录的新增/编辑/删除、模块正确率、每日趋势、8 周进步曲线、能力雷达。 */
'use strict';

Router.register('practice', {
  _module: '',

  async render() {
    const [listData, radarData, weeklyData] = await Promise.all([
      API.get('/api/practice'),
      API.get('/api/stats/radar'),
      API.get('/api/stats/weekly?weeks=8'),
    ]);
    this._all = listData.items;
    this._radar = radarData.radar;
    this._weekly = weeklyData.weeks;
    return `
    <div class="page-head">
      <div>
        <div class="page-title">笔试练习</div>
        <div class="page-desc">记录每一次刷题，用数据找到薄弱模块</div>
      </div>
      <div class="page-actions">
        <button class="btn btn-primary" id="practice-add">+ 记录一次练习</button>
      </div>
    </div>

    <div class="grid grid-2">
```

```
    <div class="card">
      <div class="card-title">🎯 能力雷达（各模块熟练度）</div>
      ${this.radarSVG()}
      <div class="radar-legend" style="margin-top:10px">
        熟练度 = 正确率 × 题量系数（200 题为满系数），帮你定位最该补的模块。
      </div>
    </div>
    <div class="card">
      <div class="card-title">📈 近 8 周进步曲线（每周题量）</div>
      ${this.weeklyBars()}
    </div>
  </div>

  <div class="card">
    <div class="filter-bar">
      <select id="practice-module-filter" class="form-select">
        <option value="">全部模块</option>
        ${Util.options(AppState.PRACTICE_MODULES, '')}
      </select>
      <span style="font-size:13px;color:#6b7a90" id="practice-count"></span>
    </div>
    <div class="table-wrap"><table class="data">
      <thead><tr>
        <th>练习日期</th><th>模块</th><th>题量</th><th>答对</th>
        <th>正确率</th><th>用时(分钟)</th><th>备注</th><th>操作</th>
      </tr></thead>
      <tbody id="practice-tbody"></tbody>
    </table></div>
    <div id="practice-empty"></div>
  </div>`;
},

afterRender(root) {
  this.renderTable();
  root.querySelector('#practice-add').addEventListener('click', () => this.openForm());
  root.querySelector('#practice-module-filter').addEventListener('change', e => {
    this._module = e.target.value;
    this.renderTable();
  });
},

/* ----- 六边形雷达图 ----- */
radarSVG() {
  const mods = this._radar;
  const size = 300, cx = 150, cy = 150, R = 105;
  const angle = i => (Math.PI * 2 * i) / mods.length - Math.PI / 2;
  const pt = (i, r) => {
    const a = angle(i);
    return (cx + r * Math.cos(a)).toFixed(1) + ',' +
           (cy + r * Math.sin(a)).toFixed(1);
  };
  // 背景网格（4 层）
  let gridPolys = '';
  [0.25, 0.5, 0.75, 1].forEach(f => {
    gridPolys += '<polygon points="' + mods.map((_, i) => pt(i, R * f)).join(' ') +
      '" fill="none" stroke="#e3e8f0" stroke-width="1"/>';
  });
  // 轴线与标签
  let axes = '', labels = '';
  mods.forEach((m, i) => {
    axes += `<line x1="${cx}" y1="${cy}" x2="${pt(i, R).split(',')[0]}" ` +
```

```
            `y2="${pt(i, R).split(',')[1]}" stroke="#e3e8f0"/>`;
      const a = angle(i);
      const lx = cx + (R + 26) * Math.cos(a), ly = cy + (R + 26) * Math.sin(a);
      labels += `<text x="${lx.toFixed(1)}" y="${ly.toFixed(1)}" text-anchor="middle" ` +
                `dominant-baseline="middle" font-size="12" fill="#243447">${m.module}</text>`;
    });
    // 数据多边形
    const dataPts = mods.map((m, i) => pt(i, R * m.proficiency)).join(' ');
    const dots = mods.map((m, i) => {
      const [dx, dy] = pt(i, R * m.proficiency).split(',');
      return `<circle cx="${dx}" cy="${dy}" r="4" fill="#f0852c">` +
        `<title>${m.module}: 正确率${Util.pct(m.accuracy)}，累计${m.total}题</title></circle>`;
    }).join('');
    return `<div class="radar-wrap"><svg class="radar-svg" viewBox="0 0 ${size} ${size}">
      ${gridPolys}${axes}
      <polygon points="${dataPts}" fill="rgba(240,133,44,.25)"
               stroke="#f0852c" stroke-width="2"/>
      ${dots}${labels}
    </svg>
    <ul class="radar-legend">` + mods.map((m, i) => {
      const color = ['#f0852c', '#3d7bd6', '#22a06b', '#6a4fc7', '#e05252', '#e6a23c'][i % 6];
      return `<li><span class="radar-dot" style="background:${color}"></span>` +
        `${m.module}: ${Util.pct(m.accuracy)}（${m.total}题）</li>`;
    }).join('') + '</ul></div>';
  },

  /* ----- 周进步条形图 ----- */
  weeklyBars() {
    const weeks = this._weekly;
    if (!weeks.length) return '<div class="empty">暂无数据</div>';
    const maxV = Math.max(1, ...weeks.map(w => w.total));
    return '<div class="bar-chart">' + weeks.map(w => {
      const width = Math.max(4, Math.round((w.total / maxV) * 100));
      return `<div class="bar-row">
        <div>${Util.esc(w.week_start.slice(5))} 当周</div>
        <div class="bar-track"><div class="bar-fill" style="width:${width}%"></div></div>
        <div class="bar-val">${w.total}题 · ${Util.pct(w.accuracy)}</div>
      </div>`;
    }).join('') + '</div>';
  },

  /* ----- 记录列表 ----- */
  renderTable() {
    const rows = this._all.filter(s => !this._module || s.module === this._module);
    const tb = document.getElementById('practice-tbody');
    document.getElementById('practice-count').textContent = '共 ' + rows.length + ' 条';
    document.getElementById('practice-empty').innerHTML =
      rows.length ? '' : '<div class="empty"><div class="big">✍</div>还没有练习记录，记录一次开始积
累吧</div>';
    tb.innerHTML = rows.map(s => {
      const acc = s.question_count ? s.correct_count / s.question_count : 0;
      const cls = acc >= 0.8 ? 'green' : (acc >= 0.6 ? 'blue' : 'orange');
      return `<tr>
        <td>${Util.md(s.practiced_at)}</td>
        <td><span class="badge purple">${Util.esc(s.module)}</span></td>
        <td>${s.question_count}</td>
        <td>${s.correct_count}</td>
        <td><span class="badge ${cls}">${Util.pct(acc)}</span></td>
        <td>${s.duration_min || '—'}</td>
        <td style="max-width:200px;overflow:hidden;text-overflow:ellipsis;white-space:nowrap"
            title="${Util.esc(s.notes || '')}">${Util.esc(s.notes || '—')}</td>
        <td class="table-ops">
```

```
        <button class="link-btn" data-act="edit" data-id="${s.id}">编辑</button>
        <button class="link-btn danger" data-act="del" data-id="${s.id}">删除</button>
      </td>
    </tr>`;
  }).join('');
  tb.querySelectorAll('button').forEach(btn => {
    const id = parseInt(btn.dataset.id, 10);
    btn.addEventListener('click', () => {
      if (btn.dataset.act === 'edit') this.openForm(id);
      else this.confirmDelete(id);
    });
  });
},

/* ----- 新增 / 编辑 ----- */
openForm(id) {
  const item = id ? this._all.find(s => s.id === id) : null;
  Modal.form(item ? '编辑练习记录' : '记录一次练习', [
    { name: 'module', label: '练习模块', type: 'select',
      options: AppState.PRACTICE_MODULES,
      value: item ? item.module : '图形推理' },
    { name: 'question_count', label: '练习题量', type: 'number', required: true,
      value: item ? item.question_count : '20', placeholder: '本次共做多少题' },
    { name: 'correct_count', label: '答对题数', type: 'number', required: true,
      value: item ? item.correct_count : '', placeholder: '答对多少题' },
    { name: 'duration_min', label: '用时（分钟）', type: 'number',
      value: item ? (item.duration_min || '') : '', placeholder: '可选' },
    { name: 'practiced_at', label: '练习日期', type: 'date',
      value: item ? (item.practiced_at || '') : Util.today() },
    { name: 'notes', label: '复盘备注', type: 'textarea',
      value: item ? item.notes : '', placeholder: '错题类型、心得体会…' },
  ], async data => {
    try {
      if (item) await API.put('/api/practice/' + id, data);
      else await API.post('/api/practice', data);
      Toast.show('练习记录已保存');
      await this.refresh();
    } catch (e) { Toast.show(e.message); return false; }
  });
},

confirmDelete(id) {
  Modal.confirm('删除确认', '确定删除这条练习记录吗？此操作不可恢复。',
    async () => {
      try {
        await API.del('/api/practice/' + id);
        Toast.show('已删除');
        await this.refresh();
      } catch (e) { Toast.show(e.message); return false; }
    });
},

async refresh() {
  const [listData, radarData, weeklyData] = await Promise.all([
    API.get('/api/practice'),
    API.get('/api/stats/radar'),
    API.get('/api/stats/weekly?weeks=8'),
  ]);
  this._all = listData.items;
  this._radar = radarData.radar;
  this._weekly = weeklyData.weeks;
```

```
    Router.render();
  },
});
```

——— static/js/view_materials.js ———

```
/* 求职备考管理软件 V1.0 · 备考资料视图
   功能：资料库的卡片展示、分类/状态筛选、搜索、新增/编辑/删除、学习进度。 */
'use strict';

Router.register('materials', {
  _category: '',
  _status: '',
  _keyword: '',

  async render() {
    const [listData, progressData] = await Promise.all([
      API.get('/api/materials'),
      API.get('/api/stats/material_progress'),
    ]);
    this._all = listData.items;
    this._progress = progressData.progress;
    return `
    <div class="page-head">
      <div>
        <div class="page-title">备考资料</div>
        <div class="page-desc">书籍、讲义、真题、课程…统一管理学习进度</div>
      </div>
      <div class="page-actions">
        <button class="btn btn-primary" id="material-add">＋ 新增资料</button>
      </div>
    </div>

    <div class="card">
      <div class="card-title">📊 分类学习进度</div>
      <div class="bar-chart" id="mat-progress"></div>
    </div>

    <div class="card">
      <div class="filter-bar">
        <select id="mat-category-filter" class="form-select">
          <option value="">全部分类</option>
          ${Util.options(AppState.MATERIAL_CATEGORIES, '')}
        </select>
        <select id="mat-status-filter" class="form-select">
          <option value="">全部状态</option>
          ${Util.options(AppState.MATERIAL_STATUS, '')}
        </select>
        <input id="mat-keyword" class="form-input search-input" placeholder="搜索标题 / 备注">
        <span style="font-size:13px;color:#6b7a90" id="mat-count"></span>
      </div>
      <div class="material-grid" id="mat-grid"></div>
      <div id="mat-empty"></div>
    </div>`;
  },

  afterRender(root) {
    this.renderProgress();
    this.renderGrid();
    root.querySelector('#material-add').addEventListener('click', () => this.openForm());
    root.querySelector('#mat-category-filter').addEventListener('change', e => {
      this._category = e.target.value; this.renderGrid();
    });
```

```
    root.querySelector('#mat-status-filter').addEventListener('change', e => {
      this._status = e.target.value; this.renderGrid();
    });
    const kw = root.querySelector('#mat-keyword');
    kw.addEventListener('input', Util.debounce(() => {
      this._keyword = kw.value.trim(); this.renderGrid();
    }));
  },

  renderProgress() {
    const el = document.getElementById('mat-progress');
    if (!this._progress.length) {
      el.innerHTML = '<div class="empty">暂无数据</div>';
      return;
    }
    el.innerHTML = this._progress.map(p => {
      const width = Math.round(p.rate * 100);
      return `<div class="bar-row">
        <div>${Util.esc(p.category)}</div>
        <div class="bar-track"><div class="bar-fill green" style="width:${width}%"></div></div>
        <div class="bar-val">${p.done}/${p.total} · ${Util.pct(p.rate)}</div>
      </div>`;
    }).join('');
  },

  filtered() {
    return this._all.filter(m => {
      if (this._category && m.category !== this._category) return false;
      if (this._status && m.status !== this._status) return false;
      if (this._keyword) {
        const kw = this._keyword.toLowerCase();
        if (!((m.title + ' ' + (m.notes || '')).toLowerCase().includes(kw))) return false;
      }
      return true;
    });
  },

  renderGrid() {
    const rows = this.filtered();
    const grid = document.getElementById('mat-grid');
    document.getElementById('mat-count').textContent = '共 ' + rows.length + ' 份';
    document.getElementById('mat-empty').innerHTML =
      rows.length ? '' : '<div class="empty"><div class="big">📚</div>还没有资料，新增第一份备考资料
吧</div>';
    grid.innerHTML = rows.map(m => `
      <div class="material-card">
        <div class="material-title">${Util.esc(m.title)}</div>
        <div class="material-meta">
          <span class="badge blue">${Util.esc(m.category)}</span>
          <span class="badge gray">${Util.esc(m.mtype || '书籍')}</span>
          ${Util.statusBadge(m.status)}
        </div>
        ${m.location ? '<div class="material-meta">📍 ' + Util.esc(m.location) + '</div>' : ''}
        <div>${Util.stars(m.rating)}</div>
        ${m.notes ? '<div class="material-notes">' + Util.esc(m.notes) + '</div>' : ''}
        <div style="margin-top:6px">
          <button class="link-btn" data-act="edit" data-id="${m.id}">编辑</button>
          <button class="link-btn" data-act="status" data-id="${m.id}">更新状态</button>
          <button class="link-btn danger" data-act="del" data-id="${m.id}">删除</button>
        </div>
      </div>`).join('');
    grid.querySelectorAll('button').forEach(btn => {
```

```
    const id = parseInt(btn.dataset.id, 10);
    btn.addEventListener('click', () => {
      const act = btn.dataset.act;
      if (act === 'edit') this.openForm(id);
      else if (act === 'del') this.confirmDelete(id);
      else if (act === 'status') this.cycleStatus(id);
    });
  });
},

// 一键流转学习状态：未开始 → 学习中 → 已完成
async cycleStatus(id) {
  const item = this._all.find(m => m.id === id);
  if (!item) return;
  const order = AppState.MATERIAL_STATUS;
  const next = order[(order.indexOf(item.status) + 1) % order.length];
  try {
    await API.put('/api/materials/' + id, { status: next });
    Toast.show('状态已更新为：' + next);
    await this.refresh();
  } catch (e) { Toast.show(e.message); }
},

openForm(id) {
  const item = id ? this._all.find(m => m.id === id) : null;
  Modal.form(item ? '编辑资料' : '新增备考资料', [
    { name: 'title', label: '资料标题', required: true,
      value: item ? item.title : '', placeholder: '如：行测模块宝典（2026版）' },
    { name: 'category', label: '资料分类', type: 'select',
      options: AppState.MATERIAL_CATEGORIES,
      value: item ? item.category : '行测' },
    { name: 'mtype', label: '资料类型', type: 'select',
      options: ['书籍', '讲义', '真题', '课程', '文档', '视频', '其他'],
      value: item ? (item.mtype || '书籍') : '书籍' },
    { name: 'location', label: '存放位置', value: item ? item.location : '',
      placeholder: '如：书架第一层 / D:/备考/讲义' },
    { name: 'status', label: '学习状态', type: 'select',
      options: AppState.MATERIAL_STATUS,
      value: item ? item.status : '未开始' },
    { name: 'rating', label: '推荐指数（0-5）', type: 'number',
      value: item ? item.rating : '0', placeholder: '0 表示未评分' },
    { name: 'notes', label: '备注', type: 'textarea',
      value: item ? item.notes : '', placeholder: '使用心得、配套资源…' },
  ], async data => {
    try {
      if (item) await API.put('/api/materials/' + id, data);
      else await API.post('/api/materials', data);
      Toast.show(item ? '已保存修改' : '资料已加入资料库');
      await this.refresh();
    } catch (e) { Toast.show(e.message); return false; }
  });
},

confirmDelete(id) {
  const item = this._all.find(m => m.id === id);
  Modal.confirm('删除确认', '确定删除资料「' + item.title + '」吗？此操作不可恢复。',
    async () => {
      try {
        await API.del('/api/materials/' + id);
        Toast.show('已删除');
        await this.refresh();
```

```
        } catch (e) { Toast.show(e.message); return false; }
      });
  },

  async refresh() {
    const [listData, progressData] = await Promise.all([
      API.get('/api/materials'),
      API.get('/api/stats/material_progress'),
    ]);
    this._all = listData.items;
    this._progress = progressData.progress;
    this.renderProgress();
    this.renderGrid();
  },
});
```

——— static/js/view_todos.js ———

```
/* 求职备考管理软件 V1.0 · 待办清单视图
   功能：待办的新增/编辑/删除、完成切换、「今日三件事」标记、逾期提醒、分类筛选。 */
'use strict';

Router.register('todos', {
  _category: '',
  _showDone: true,

  async render() {
    const data = await API.get('/api/todos');
    this._all = data.items;
    const undone = data.items.filter(t => !t.done).length;
    const overdue = data.items.filter(t => {
      if (t.done || !t.due_date) return false;
      return t.due_date < Util.today();
    }).length;
    return `
    <div class="page-head">
      <div>
        <div class="page-title">待办清单</div>
        <div class="page-desc">把脑子里的事倒出来，每天只盯最重要的三件</div>
      </div>
      <div class="page-actions">
        <button class="btn btn-primary" id="todo-add">＋ 新增待办</button>
      </div>
    </div>

    ${overdue > 0 ? `<div class="card" style="border-left:4px solid #e05252">
      <div class="card-title" style="color:#e05252">⚠️ 有 ${overdue} 项待办已逾期</div>
      <div style="font-size:13px;color:#6b7a90">逾期的事项已在下方用红色标记，建议优先处理或调整截止日
期。</div>
    </div>` : ''}

    <div class="card">
      <div class="card-title">⭐ 今日三件事</div>
      <div id="top3-list"></div>
    </div>

    <div class="card">
      <div class="filter-bar">
        <select id="todo-category-filter" class="form-select">
          <option value="">全部分类</option>
          ${Util.options(AppState.TODO_CATEGORIES, '')}
        </select>
        <label style="font-size:13px;display:flex;align-items:center;gap:6px">
          <input type="checkbox" id="todo-show-done" checked> 显示已完成
```

```
      </label>
      <span style="font-size:13px;color:#6b7a90" id="todo-count"></span>
    </div>
    <div id="todo-list"></div>
  </div>`;
},

afterRender(root) {
  this.renderTop3();
  this.renderList();
  root.querySelector('#todo-add').addEventListener('click', () => this.openForm());
  root.querySelector('#todo-category-filter').addEventListener('change', e => {
    this._category = e.target.value;
    this.renderList();
  });
  root.querySelector('#todo-show-done').addEventListener('change', e => {
    this._showDone = e.target.checked;
    this.renderList();
  });
},

isOverdue(t) {
  return !t.done && t.due_date && t.due_date < Util.today();
},

renderTop3() {
  const el = document.getElementById('top3-list');
  const top3 = this._all.filter(t => t.is_top3 && !t.done).slice(0, 3);
  if (!top3.length) {
    el.innerHTML = '<div class="empty" style="padding:20px">' +
      '还没有标记今日三件事 — 点击待办右侧的「设为今日三件事」开始聚焦</div>';
    return;
  }
  el.innerHTML = top3.map((t, i) => `
    <div class="todo-item">
      <div style="font-size:20px">${['🥇', '🥈', '🥉'][i]}</div>
      <div class="todo-main">
        <div class="todo-title">${Util.esc(t.title)}</div>
        <div class="todo-meta">${this.metaHTML(t)}</div>
      </div>
      <button class="btn btn-sm btn-accent" data-id="${t.id}" data-act="done">完成</button>
    </div>`).join('');
  el.querySelectorAll('button').forEach(b =>
    b.addEventListener('click', () => this.toggleDone(parseInt(b.dataset.id, 10))));
},

metaHTML(t) {
  const parts = [`<span>${Util.esc(t.category)}</span>`];
  if (t.due_date) {
    parts.push(this.isOverdue(t)
      ? '<span class="overdue-tag">逾期: ' + Util.md(t.due_date) + '</span>'
      : '<span>截止 ' + Util.md(t.due_date) + '</span>');
  }
  if (t.is_top3 && !t.done) parts.push('<span class="top3-flag">★ 今日三件事</span>');
  return parts.join('');
},

renderList() {
  const el = document.getElementById('todo-list');
  const rows = this._all.filter(t => {
    if (this._category && t.category !== this._category) return false;
```

```
      if (!this._showDone && t.done) return false;
      return true;
    });
    // 排序：未完成优先 → 今日三件事 → 逾期 → 优先级 → 截止日期
    const pw = { '高': 0, '中': 1, '低': 2 };
    rows.sort((a, b) =>
      (a.done - b.done) || ((b.is_top3 || 0) - (a.is_top3 || 0)) ||
      ((this.isOverdue(b) ? 1 : 0) - (this.isOverdue(a) ? 1 : 0)) ||
      ((pw[a.priority] || 1) - (pw[b.priority] || 1)));
    document.getElementById('todo-count').textContent =
      '共 ' + rows.length + ' 项（未完成 ' +
      rows.filter(t => !t.done).length + '）';
    if (!rows.length) {
      el.innerHTML = '<div class="empty"><div class="big">✅</div>干干净净，没有待办</div>';
      return;
    }
    el.innerHTML = rows.map(t => `
      <div class="todo-item ${t.done ? 'done' : ''}">
        <div class="todo-check ${t.done ? 'on' : ''}" data-id="${t.id}" data-act="toggle">
          ${t.done ? '✓' : ''}</div>
        <div class="todo-main">
          <div class="todo-title">${Util.esc(t.title)}</div>
          <div class="todo-meta">${this.metaHTML(t)} ${Util.priorityBadge(t.priority)}</div>
        </div>
        <div class="todo-ops">
          ${!t.is_top3 && !t.done
            ? `<button class="link-btn" data-id="${t.id}" data-act="top3">设为今日三件
事</button>` : ''}
          ${t.is_top3 ? `<button class="link-btn" data-id="${t.id}" data-act="untop3">取消标
记</button>` : ''}
          <button class="link-btn" data-id="${t.id}" data-act="edit">编辑</button>
          <button class="link-btn danger" data-id="${t.id}" data-act="del">删除</button>
        </div>
      </div>`).join('');
    el.querySelectorAll('[data-act]').forEach(node => {
      const id = parseInt(node.dataset.id, 10);
      const act = node.dataset.act;
      node.addEventListener('click', () => {
        if (act === 'toggle') this.toggleDone(id);
        else if (act === 'edit') this.openForm(id);
        else if (act === 'del') this.confirmDelete(id);
        else if (act === 'top3' || act === 'untop3') this.setTop3(id, act === 'top3');
      });
    });
  },

  async toggleDone(id) {
    try {
      await API.post('/api/todos/' + id + '/toggle');
      await this.refresh();
      const item = this._all.find(t => t.id === id);
      if (item && item.done) Toast.show('🎉 又完成一件，继续保持');
    } catch (e) { Toast.show(e.message); }
  },

  async setTop3(id, on) {
    try {
      await API.put('/api/todos/' + id, { is_top3: on ? 1 : 0 });
      Toast.show(on ? '已加入今日三件事' : '已取消标记');
      await this.refresh();
    } catch (e) { Toast.show(e.message); }
  },
```

```
  openForm(id) {
    const item = id ? this._all.find(t => t.id === id) : null;
    const fields = [
      { name: 'title', label: '待办标题', required: true,
        value: item ? item.title : '', placeholder: '如：完成蓝桥软件笔试' },
      { name: 'category', label: '分类', type: 'select',
        options: AppState.TODO_CATEGORIES, value: item ? item.category : '其他' },
      { name: 'priority', label: '优先级', type: 'select',
        options: AppState.PRIORITIES, value: item ? item.priority : '中' },
      { name: 'due_date', label: '截止日期', type: 'date',
        value: item ? (item.due_date || '') : '' },
    ];
    if (!item) {
      fields.push({ name: 'is_top3', label: '', type: 'checkbox',
                    value: 0, checkLabel: '直接加入今日三件事' });
    }
    Modal.form(item ? '编辑待办' : '新增待办', fields, async data => {
      try {
        if (item) await API.put('/api/todos/' + id, data);
        else await API.post('/api/todos', data);
        Toast.show(item ? '已保存修改' : '待办已加入清单');
        await this.refresh();
      } catch (e) { Toast.show(e.message); return false; }
    });
  },

  confirmDelete(id) {
    Modal.confirm('删除确认', '确定删除这条待办吗？此操作不可恢复。',
      async () => {
        try {
          await API.del('/api/todos/' + id);
          Toast.show('已删除');
          await this.refresh();
        } catch (e) { Toast.show(e.message); return false; }
      });
  },

  async refresh() {
    const data = await API.get('/api/todos');
    this._all = data.items;
    Router.render();
  },
});
```

——— static/js/view_exams.js ———

```
/* 求职备考管理软件 V1.0 · 考试日程视图
   功能：笔试 / 面试等关键时间节点的增删改查与倒计时展示。 */
'use strict';

Router.register('exams', {
  async render() {
    const data = await API.get('/api/exams');
    this._all = data.items;
    return `
    <div class="page-head">
      <div>
        <div class="page-title">考试日程</div>
        <div class="page-desc">笔试、面试…关键时间节点一个都不漏</div>
      </div>
      <div class="page-actions">
        <button class="btn btn-primary" id="exam-add">＋ 新增考试</button>
      </div>
```

```
    </div>
    <div class="card">
      <div class="card-title">🗓️ 全部日程（按日期排序）</div>
      <div id="exam-list"></div>
    </div>`;
  },

  afterRender(root) {
    this.renderList();
    root.querySelector('#exam-add').addEventListener('click', () => this.openForm());
  },

  renderList() {
    const el = document.getElementById('exam-list');
    if (!this._all.length) {
      el.innerHTML = '<div class="empty"><div class="big">📅</div>还没有考试安排，新增一个开始倒计时吧
</div>';
      return;
    }
    el.innerHTML = this._all.map(e => {
      const d = /^(\d{4})-(\d{2})-(\d{2})/.exec(e.exam_date) || [];
      const urgent = e.days_left !== null && e.days_left >= 0 && e.days_left <= 7;
      const passed = e.days_left !== null && e.days_left < 0;
      const cdText = e.days_left === null ? '日期未知'
        : passed ? '已过去 ' + (-e.days_left) + ' 天'
        : e.days_left === 0 ? '就是今天! '
        : '还剩 ' + e.days_left + ' 天';
      return `<div class="exam-card" style="padding:12px 0;border-bottom:1px solid #e3e8f0">
        <div class="exam-date-box ${urgent ? 'urgent' : ''}">
          <div class="d">${d[3] ? parseInt(d[3], 10) : '—'}</div>
          <div class="m">${d[2] ? parseInt(d[2], 10) + '月' : ''}</div>
        </div>
        <div class="exam-info">
          <div class="exam-name">${Util.esc(e.name)}</div>
          <div class="exam-meta">
            ${e.location ? '📍 ' + Util.esc(e.location) + '  ' : ''}
            ${e.registration_deadline ? '报名截止 ' + Util.md(e.registration_deadline) + '  ' :
''}
          </div>
          ${e.notes ? '<div class="exam-meta">📝 ' + Util.esc(e.notes) + '</div>' : ''}
        </div>
        <div class="countdown ${passed ? 'passed' : ''}">${cdText}</div>
        <div class="todo-ops">
          <button class="link-btn" data-act="edit" data-id="${e.id}">编辑</button>
          <button class="link-btn danger" data-act="del" data-id="${e.id}">删除</button>
        </div>
      </div>`;
    }).join('');
    el.querySelectorAll('button').forEach(btn => {
      const id = parseInt(btn.dataset.id, 10);
      btn.addEventListener('click', () => {
        if (btn.dataset.act === 'edit') this.openForm(id);
        else this.confirmDelete(id);
      });
    });
  },

  openForm(id) {
    const item = id ? this._all.find(e => e.id === id) : null;
    Modal.form(item ? '编辑考试' : '新增考试日程', [
      { name: 'name', label: '考试名称', required: true,
        value: item ? item.name : '', placeholder: '如: 国家公务员考试（笔试）' },
      { name: 'exam_date', label: '考试日期', type: 'date', required: true,
```

```
      value: item ? (item.exam_date || '') : '' },
    { name: 'location', label: '考试地点', value: item ? item.location : '',
      placeholder: '如：合肥考区 / 线上' },
    { name: 'registration_deadline', label: '报名截止日期', type: 'date',
      value: item ? (item.registration_deadline || '') : '' },
    { name: 'notes', label: '备注', type: 'textarea',
      value: item ? item.notes : '', placeholder: '考试科目、携带物品…' },
  ], async data => {
    try {
      if (item) await API.put('/api/exams/' + id, data);
      else await API.post('/api/exams', data);
      Toast.show(item ? '已保存修改' : '考试日程已新增');
      await this.refresh();
    } catch (e) { Toast.show(e.message); return false; }
  });
},

confirmDelete(id) {
  Modal.confirm('删除确认', '确定删除这个考试日程吗？此操作不可恢复。',
    async () => {
      try {
        await API.del('/api/exams/' + id);
        Toast.show('已删除');
        await this.refresh();
      } catch (e) { Toast.show(e.message); return false; }
    });
},

async refresh() {
  const data = await API.get('/api/exams');
  this._all = data.items;
  this.renderList();
},
});
```

——— static/js/view_settings.js ———

```
/* 求职备考管理软件 V1.0 · 设置视图
   功能：修改密码、演示数据管理、数据备份导出 / 恢复导入、关于。 */
'use strict';

Router.register('settings', {
  async render() {
    return `
    <div class="page-head">
      <div>
        <div class="page-title">设置</div>
        <div class="page-desc">账号安全与数据管理</div>
      </div>
    </div>

    <div class="card">
      <div class="card-title">🔐 账号安全</div>
      <div class="setting-row">
        <div>
          <div class="setting-label">修改登录密码</div>
          <div class="setting-desc">定期更换密码，保护本地数据安全</div>
        </div>
        <button class="btn" id="btn-change-pwd">修改密码</button>
      </div>
      <div class="setting-row">
        <div>
          <div class="setting-label">退出登录</div>
```

```
        <div class="setting-desc">清除当前浏览器会话</div>
      </div>
      <button class="btn" id="btn-logout">退出登录</button>
    </div>
  </div>

  <div class="card">
    <div class="card-title">💾 数据管理</div>
    <div class="setting-row">
      <div>
        <div class="setting-label">写入演示数据</div>
        <div class="setting-desc">一键填入示例数据，快速体验全部功能（会先清空现有数据）</div>
      </div>
      <button class="btn" id="btn-seed">写入演示数据</button>
    </div>
    <div class="setting-row">
      <div>
        <div class="setting-label">导出数据备份</div>
        <div class="setting-desc">将全部业务数据导出为 JSON 文件，妥善保存</div>
      </div>
      <button class="btn" id="btn-backup">导出备份</button>
    </div>
    <div class="setting-row">
      <div>
        <div class="setting-label">从备份恢复</div>
        <div class="setting-desc">选择之前导出的 JSON 备份文件进行恢复（会覆盖当前数据）</div>
      </div>
      <button class="btn" id="btn-restore">选择文件恢复</button>
      <input type="file" id="restore-file" accept=".json,application/json" class="hidden">
    </div>
  </div>

  <div class="card">
    <div class="card-title">ℹ️ 关于软件</div>
    <div class="about-box">
      <b>求职备考管理软件 V1.0</b><br>
      为求职季与备考季打造的本地化管理工具，覆盖求职投递跟踪、笔试练习记录、<br>
      备考资料管理、待办清单与考试日程五大场景，数据仅保存在本机，无需联网。<br>
      技术栈：Python Flask + SQLite + 原生 HTML / CSS / JavaScript。<br>
      开发完成日期：2026 年 10 月。
    </div>
  </div>`;
},

afterRender(root) {
  root.querySelector('#btn-change-pwd').addEventListener('click', () => {
    Modal.form('修改密码', [
      { name: 'old_password', label: '原密码', type: 'password', required: true },
      { name: 'new_password', label: '新密码（至少 6 位）', type: 'password', required: true },
      { name: 'new_password2', label: '确认新密码', type: 'password', required: true },
    ], async data => {
      if (data.new_password !== data.new_password2) {
        Toast.show('两次输入的新密码不一致');
        return false;
      }
      try {
        await API.post('/api/change_password', {
          old_password: data.old_password,
          new_password: data.new_password,
        });
        Toast.show('密码修改成功，请牢记新密码');
```

```
      } catch (e) { Toast.show(e.message); return false; }
    });
  });

  root.querySelector('#btn-logout').addEventListener('click', async () => {
    await API.post('/api/logout');
    AppState.user = null;
    Router.go('login');
  });

  root.querySelector('#btn-seed').addEventListener('click', () => {
    Modal.open('写入演示数据',
      '<p style="font-size:14px">将清空当前全部业务数据并写入演示数据，确定继续吗？</p>',
      async () => {
        try {
          await API.post('/api/seed_demo');
          Toast.show('演示数据已写入');
        } catch (e) { Toast.show(e.message); return false; }
      });
  });

  root.querySelector('#btn-backup').addEventListener('click', async () => {
    try {
      const data = await API.get('/api/backup');
      const blob = new Blob([JSON.stringify(data.backup, null, 2)],
                            { type: 'application/json' });
      const a = document.createElement('a');
      a.href = URL.createObjectURL(blob);
      a.download = '求职备考管理软件_备份_' + Util.today() + '.json';
      document.body.appendChild(a);
      a.click();
      document.body.removeChild(a);
      URL.revokeObjectURL(a.href);
      Toast.show('备份文件已开始下载');
    } catch (e) { Toast.show(e.message); }
  });

  const fileInput = root.querySelector('#restore-file');
  root.querySelector('#btn-restore').addEventListener('click', () => fileInput.click());
  fileInput.addEventListener('change', () => {
    const file = fileInput.files[0];
    if (!file) return;
    const reader = new FileReader();
    reader.onload = () => {
      let backup = null;
      try { backup = JSON.parse(reader.result); }
      catch (e) { Toast.show('备份文件格式不正确'); return; }
      Modal.open('从备份恢复',
        '<p style="font-size:14px">将用备份文件覆盖当前全部业务数据，确定继续吗？</p>',
        async () => {
          try {
            await API.post('/api/restore', { backup });
            Toast.show('数据恢复成功');
          } catch (e) { Toast.show(e.message); return false; }
        });
    };
    reader.readAsText(file);
    fileInput.value = '';
  });
 },
});
```